Vermont Health Platform — Documentation

Vermont Health Platform

Complete Documentation Set

Healthcare Transformation Roadmap (HTR)

Technical · Operations · Content · User Guide

A full reference covering architecture, local development, content creation (Sanity & Supabase), the Academy, the AI Analyst & RAG backend, tooling, deployment & maintenance, the end-user guide, and reference appendices.

Verified against the live codebase.

Generated 2026-06-06

Vermont Health Platform — Complete Documentation Set

> **Audience:** Engineers, operators, content editors, and end-users of the Vermont Health Platform (internally "HTR" — *Healthcare Transformation Roadmap*). **Status:** Authoritative reference, verified against the live codebase. Where this set and older ad-hoc `.md` files in the repo root disagree, **this set wins. Last verified against code:** see each document header.

What this platform is

The Vermont Health Platform is a full-stack web application that publishes healthcare-transformation research, runs an online learning **Academy**, hosts interactive policy simulators and dashboards, and provides a retrieval-augmented (RAG) **AI Analyst**. It is organized around a **Six-Pillar Framework**: **Policy, Economics, Technology, Clinical, Equity, Operations.**

It is built from three cooperating systems:

System	Technology	Responsibility
Frontend / web app	Next.js 16 (App Router), React 19, TypeScript, Tailwind v4	All pages, UI, route handlers (`/api/*`), Stripe, auth session, Sanity Studio mount
CMS	Sanity (project `fxz10xl7`, dataset `production`)	Editorial content: Analyses, Courses, Case Studies, Webinars, Reports, Glossary, Ticker, etc.
Application database	Supabase (Postgres + Auth + Storage + pgvector)	Users, roles, subscriptions, course progress, bookmarks, community, RAG vectors
AI Brain (backend)	FastAPI (Python), LlamaIndex, Groq/Anthropic/OpenAI	RAG chat, suggestions, content ingest, embeddings, Vermont-ops tools

---

How to read this set

Read in order if you are new. Jump by role if you are not.

#	Document	Read this if you are…
01	Architecture & System Overview	An engineer onboarding, or anyone who needs the big picture
02	Local Development & Environment Setup	Setting up the app on a new machine
03	Content Creation — Sanity CMS	A content editor or author publishing Analyses, Courses, etc.
04	Content & Data — Supabase	Managing users, roles, course enrollment, the database
05	The Academy — Courses, Lessons, Quizzes, Certificates	Building or maintaining course content
06	The AI Analyst & RAG Backend	Operating or extending the AI chat / ingest pipeline
07	Tooling & Scripts Reference	Running any of the maintenance / seed / content scripts
08	Operations, Deployment & Maintenance	Deploying, monitoring, upgrading, or doing incident response
09	User Guide & Usability	An end-user, or writing user-facing help
10	Reference Appendices	Anyone needing env-var, route, schema, or table lookups

---

The Six-Pillar Framework (the spine of everything)

Almost every content type, navigation section, and dashboard is keyed to one of six pillars. Memorize these — they appear as a `pillar` field on Sanity documents, as top-level nav sections, and as Research-Lab workspaces.

Pillar	What it covers	Example subcategories
Policy	Regulation, legislation, mandates, feasibility	Regulation & Legislation, Public Health Mandates, Global & Comparative Policy, Policy Feasibility Studies
Economics	Value-based care, market/finance, workforce, investment	Value-Based Care Models, Market & Finance, Labor & Workforce Strategy, Healthcare Investment Trends
Technology	AI/ML, digital health, security, workflow	AI & Machine Learning, Digital Health & Telemedicine, Data Security & Governance, Tech-Enabled Workflow
Clinical	Care delivery models	Hospital-at-Home, Precision Medicine, Virtual Care Models, Population Health
Equity	Disparity & SDOH	SDOH Integration, Algorithmic Bias, Access Disparity, Community Engagement
Operations	Running a health system	Revenue Cycle, Supply Chain, Workforce, Compliance, Payer Network

---

Conventions used in this documentation

- **Code paths** are written relative to the repo root, e.g. `frontend/app/api/chat/route.ts`.
- **Commands** assume your shell is at the repo root unless noted (`cd frontend` is called out explicitly).
- ⚠️ **Danger** boxes mark operations that mutate production data or are hard to reverse.
- 🔑 **Secret** boxes mark places that require credentials not stored in the repo.

Document conventions for "page count"

This set is intentionally split into ten Markdown documents. Rendered to PDF/print (the repo already ships a PDF-export pipeline; see Doc 07), the set runs well past 35 pages. Each document is self-contained with its own table of contents.

01 — Architecture & System Overview

> **Verified against:** `frontend/package.json` (Next 16.1.6, React 19), `backend/main.py` (HTR AI Brain v4.2.0), `backend/requirements.txt`, `supabase/migrations/*`, `vercel.json`, `backend/fly.toml`.

Table of contents

---

1. The four runtimes

The platform is **not** a single deployable. It is four cooperating runtimes:


Browser
NEXT.JS APP (Vercel, region iad1)
- All pages (App Router, RSC + client)
- /api/* route handlers (Node runtime)
- /studio (embedded Sanity Studio)
- Stripe checkout/portal/webhook
Supabase JS (anon+SSR)
Sanity client
fetch() to BACKEND
SUPABASE
Postgres+Auth
+Storage
+pgvector
SANITY
CMS
fxz10xl7
GROQ
FASTAPI BACKEND
"HTR AI Brain"
(Fly.io, sjc)
LlamaIndex+Groq
webhook
(ingest)

Runtime	Hosted on	Entry point	Notes
Next.js web app	**Vercel** (`iad1`)	`frontend/`	Build: `cd frontend && next build` ; output `frontend/.next`
FastAPI AI Brain	**Fly.io** (`sjc` , app `vhp-backend`) — also has Railway & Procfile configs	`backend/main.py` (`app`)	`uvicorn main:app` . Auto-stop/auto-start machines, scale-to-zero
Supabase	**Supabase Cloud**	`supabase/migrations/`	Postgres 15+, Auth (GoTrue), Storage, pgvector
Sanity	**Sanity Cloud**	`frontend/sanity/` + `frontend/app/studio`	Project `fxz10xl7` , dataset `production`

> The backend has **three** deploy descriptors (`fly.toml` , `railway.toml` , `Procfile`). Fly.io is the active target — see Doc 08. The others are fallbacks.

---

2. Request lifecycle — a page load

1. Browser hits e.g. `/policy/regulation` .
2. Vercel serves the Next.js App Router. The route is a **React Server Component** by default.
3. The RSC fetches editorial content from **Sanity** via `frontend/lib/sanity-fetch.ts` (GROQ queries), and any user/personalization data from **Supabase** via the SSR client (`frontend/lib/supabase.ts`).
4. Shared chrome is composed in `frontend/components/AppShell.tsx` (header, sidebars, ticker, footer). Client interactivity (voice, AI sidebar, command palette) hydrates on the client.
5. The page streams to the browser. The **RightSidebar** AI Analyst and the **TickerStrip** (system vitals) are client components that call `/api/*` handlers after hydration.

3. Request lifecycle — an AI Analyst question

1. User types into the AI Analyst (right sidebar widget, or full page at `/chat`).
2. The client calls the Next.js route handler `frontend/app/api/chat/route.ts` , which (a) authenticates the Supabase

session and (b) proxies to the FastAPI backend `POST /api/chat` (`BACKEND_URL`).

3. Backend (`backend/routers/chat.py`) resolves the caller's **role** (free / subscriber / professional / advisory / admin). Subscribers+ get RAG; professional/advisory/admin roles get the **agentic ReAct** pipeline with tools.
4. `HybridRetriever` (vector + keyword) pulls candidate nodes from Supabase pgvector, FlashRank/`get_ranker()` reranks, Vermont-specific nodes are boosted (`boost_vermont_nodes`).
5. The LLM (Groq `llama-3.3-70b-versatile` for subscribers by default; configurable) streams a grounded answer back with citations, which is relayed to the browser as a `StreamingResponse`.

4. Content flow — Sanity → ingest → RAG

```
Editor publishes in Sanity Studio (/studio)
        │
        ▼
Sanity GROQ webhook  ──POST──►  /api/webhooks/sanity (Next.js)
        │  (filtered to: policyAnalysis, post, academyModule,
        │   caseStudy, definition, analystNote, webinar,
report)
        ▼
Backend  /api/ingest/webhook  (auth: Bearer INGEST_SECRET)
        │
        ▼
Document parsed → chunked → embedded (OpenAI embeddings)
        │
        ▼
Stored in Supabase pgvector (rag_documents / vector store)
        │
        ▼
Available to AI Analyst RAG immediately
```

This is the single most important integration to understand: **the CMS is the source of editorial truth; the RAG index is a derived, regenerable copy.** If RAG drifts from Sanity, re-run ingest (see Doc 06).

---

5. The frontend in detail

Stack: Next.js 16.1.6 (App Router, `--webpack` dev), React 19, TypeScript 5, Tailwind CSS v4 (beta PostCSS plugin), styled-components 6 (for Sanity Studio).

Key libraries (`frontend/package.json` / root `package.json`):

- `next-sanity`, `@sanity/image-url`, `@sanity/vision`, `@sanity/code-input` — CMS integration + Studio.

- `@supabase/ssr`, `@supabase/supabase-js` — auth + DB from server and client.
- `stripe`, `@stripe/stripe-js` — billing.
- `chart.js` + `react-chartjs-2`, `react-simple-maps`, `react-leaflet` + `leaflet`, `d3-geo` — dashboards, simulators, maps.
- `@sentry/nextjs` — error monitoring.
- `react-markdown` — rendering markdown content blocks.

Directory map (`frontend/`):

- `app/` — every route. ~150 page routes + ~40 API route handlers. See §10 and Doc 10 appendix.
- `components/` — ~80 shared components (`AppShell`, `Header`, `RightSidebar`, `HomeSidebar`, `AcademyContent`, `TickerStrip`, voice components, etc.) plus subfolders `academy/`, `advisory/`, `connect/`, `course/`, `dashboard/`, `policy/`, `research/`.
- `lib/` — data + integration layer: `sanity.ts`, `sanity-fetch.ts`, `supabase.ts`, `stripe.ts`, `auth.ts`, `chat.ts`, `loops.ts`, `narration.ts`, `course-api.ts`, plus `ai/`, `data/`, `db/`, `hooks/`, `context/`, `taxonomy/`.
- `sanity/` — Studio config + schema (`schemaTypes/`), content seeds, generation prompts.
- `scripts/` — ~90 maintenance/seed/content scripts (see Doc 07).
- `supabase/` — local helpers.
- `e2e/` — Playwright tests.

Notable route groups (full list in Doc 10):

- Six pillar sections: `/policy`, `/economics`, `/technology`, `/clinical`, `/equity`, `/operations` — each with `[slug]` and named subpages.
- **Academy:** `/academy/*` (courses, tracks, modules, case-studies, faculty, glossary, webinars, personalized-learning).
- **Account:** `/account/*` (profile, billing, subscription, courses, bookmarks, referrals, api-keys).
- **Admin:** `/admin/*` (users, analytics, revenue, access-codes, role-changes, ingest).
- **Advisory:** `/advisory/*` (consulting, regulatory, financial-audit, training, reports…).
- **Research Lab:** `/research-lab/*` (seven workspaces incl. `technology-ai`, `payment-models`, `population-equity`).
- **State / simulators:** `/states/[state]`, `/dashboard/[state]`, `/compare-states`, plus Vermont-specific (`/vermont-act-167`, `/vermont-act-68`, `/vermont-rht-program`, `/vermont-blueprint`, …) and multi-state simulators (`/california-calaim`, `/oregon-cco`).
- **The Wire** (`/the-wire`), **Book** (`/book`, `/book/listen`), **HTR Index/Simulator**, **System Vitals**.

6. The backend (AI Brain) in detail

`backend/main.py` is a FastAPI **application factory** (refactored from a monolith in 2026-03). Title `HTR AI Brain`, version `4.2.0`.

Module layout:

- `config.py` — all env-var constants in one place (never scatter `os.getenv`).
- `services/db.py` — Supabase singleton.
- `services/auth.py` — JWT verification (`SUPABASE_JWT_SECRET`), role gating (`require_subscriber`, etc.).
- `services/llm.py` — LLM factory (`get_llm_for_role`), FlashRank reranker (`get_ranker`), `init_global_settings`.
- `services/retrieval.py` — `HybridRetriever`, `StaticNodeRetriever`, `rerank_nodes`, `extract_citations`, `boost_vermont_nodes`.
- `services/indexing.py` — `build_index`, `load_index`, `fetch_sanity_content`.
- `services/catalog_search.py` — semantic tool/feature catalog so the AI can point users to the right page.
- `services/tools.py` — `ALL_TOOLS` for the agentic pipeline.
- `services/medicaid_parser.py`, `services/retrieval.py` — domain-specific helpers.
- **Routers:** `chat.py` (`/api/chat`, `/api/suggest`), `ingest.py` (`/api/ingest*`), `api_v1.py` (`/api/v1/*` developer API, key-authenticated), `personalized_learning.py`, `vermont_ops.py`.

Cross-cutting: Sentry (init before app code), CORS (locked to `FRONTEND_URL` + localhost), `slowapi` rate limiting, `/health` healthcheck.

LLM providers: Groq (default, `llama-3.3-70b-versatile`), Anthropic, OpenAI — selected per role. Embeddings via OpenAI. See Doc 06.

7. Data stores and what lives where

Data	Store	Why
Editorial content (Analyses, Courses, Case Studies, Webinars, Reports, Glossary, Ticker, Daily Insight, Hospital/State profiles, Investment Deals)	**Sanity**	Editor-friendly, versioned, Portable Text
Users, auth sessions, roles	**Supabase Auth** + `profiles`, `user_roles`	Identity
Subscriptions & payments	**Supabase** (`subscriptions`, `stripe_customers`, `stripe_events`) mirrored from **Stripe**	Billing source of truth is Stripe; mirrored for fast reads
Course enrollment & progress	**Supabase** (`course_enrollments`, `course_player_enrollments`, `course_lesson_progress`, `module_progress`, `course_quiz_attempts`, `certifications`)	Per-user state
Academy *structure* (courses→tracks→lessons, quizzes)	**Supabase** (`courses`, `tracks`, `lessons`, `quizzes`, …) — lesson *body* links to Sanity via `sanity_slug`	Hybrid: structure in DB, rich content in CMS
RAG vectors	**Supabase pgvector** (`rag_documents`, HNSW index)	Semantic search
Bookmarks, notes, community, referrals, API keys, survey, ticker cache	**Supabase**	App state

> **Critical academy nuance:** a lesson's prose lives in Sanity, but the lesson *row* lives in Supabase. The two are linked by setting the Supabase lesson's `sanity_slug` equal to the Sanity document's slug/`_id`. If that link is missing the app renders thin legacy `content_blocks` instead of the rich body. See Doc 05.

8. Authentication & authorization model

- **Identity:** Supabase Auth. Sessions are carried server-side via `@supabase/ssr`.
- **Roles** (ascending capability): `free` → `subscriber` → `professional` → `advisory` → `admin`. Stored in `user_roles` / `profiles`; audited in `role_change_log`.
- **Gating on the frontend:** route handlers and server components check the session + role. Paywalled content uses `ArticlePaywall` / `UpgradePrompt`.
- **Gating on the backend:** `services/auth.py` verifies the Supabase JWT (`SUPABASE_JWT_SECRET`) and enforces role with `require_subscriber` and friends. RAG chat requires subscriber+; the agentic ReAct pipeline is limited to `professional`, `advisory`, `admin`.
- **Beta access:** `beta_access_codes` table + `/api/beta/verify` gate pre-launch access.
- **Developer API:** `/api/v1/*` is authenticated by HMAC-hashed API keys (`api_keys` table, header `X-HTR-API-Key`).

9. Third-party services

Service	Used for	Key env vars
Sanity	CMS	`NEXT_PUBLIC_SANITY_PROJECT_ID`, `SANITY_API_TOKEN`, `SANITY_WEBHOOK_SECRET`
Supabase	DB/Auth/Storage/pgvector	`NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_JWT_SECRET`, `SUPABASE_DB_URL`
Stripe	Subscriptions + team checkout	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_*`
Groq / Anthropic / OpenAI	LLM + embeddings	`GROQ_API_KEY`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`
Loops	Transactional/marketing email	`LOOPS_API_KEY`, `LOOPS_TEMPLATE_*`
Sentry	Error monitoring	`SENTRY_DSN`, `SENTRY_AUTH_TOKEN`
Piper	Local TTS narration (voice)	(`.piper-venv`, `.piper-voices`)

10. Repository layout

```
Vermont-Health-Platform/
├── frontend/              # Next.js app (the deployable web
app)
│   ├── app/               # routes (pages + /api route
handlers + /studio)
│   ├── components/        # shared React components
│   ├── lib/               # data & integration layer (sanity,
supabase, stripe…)
│   ├── sanity/            # Studio config + schemaTypes +
content seeds
│   ├── scripts/           # ~90 seed/content/maintenance
scripts
│   ├── e2e/               # Playwright tests
│   └── docs/              # ← this documentation lives here
├── backend/               # FastAPI "HTR AI Brain"
│   ├── main.py            # app factory
│   ├── config.py          # env constants
│   ├── routers/           # chat, ingest, api_v1,
personalized_learning, vermont_ops
│   ├── services/          # db, auth, llm, retrieval,
indexing, tools, catalog_search
│   ├── scripts/           # CMS/CMS-data loaders (CMS
hospitals, HTI scores…)
│   ├── data/, data_rfp/   # source data
│   ├── fly.toml / railway.toml / Procfile / Dockerfile
│   └── requirements.txt
├── supabase/
│   ├── migrations/        # 34 ordered SQL migrations (001 …
033 + dated)
│   └── seed/
├── scripts/               # repo-level scripts (narration
audio, digests)
├── training/              # end-user guides & video scripts
(user-facing)
└── *.md                   # legacy planning docs (treat as
historical)
```

Continue to → 02 — Local Development & Environment Setup

02 — Local Development & Environment Setup

> **Verified against:** `frontend/package.json` scripts, `backend/requirements.txt`, `backend/main.py` run instructions, `frontend/.env.production.example`, `backend/config.py`.

Table of contents

---

1. Prerequisites

Tool	Version	Used by
Node.js	20.x (CI pins Node 20)	frontend, scripts
npm	bundled with Node 20	install/build
Python	3.13 (the working venv is `python3.13`)	backend AI Brain
Git	any recent	source control
Sanity CLI	`npx sanity` (no global install needed)	Studio admin
Stripe CLI	optional	local webhook testing
Supabase CLI	optional	local migrations
Fly CLI (`flyctl`)	optional	backend deploy

You also need accounts/credentials for: **Supabase**, **Sanity** (project `fxz10xl7`), **Stripe** (test mode), and at least one LLM key (**Groq** is the cheapest default; **OpenAI** is required for embeddings).

2. Clone and install

```
git clone <repo-url> Vermont-Health-Platform
cd Vermont-Health-Platform

# Root install (the monorepo hoists frontend deps to the root
        node_modules;
# vercel.json buildCommand also relies on
        ../node_modules/.bin/next).
npm install

# Frontend has its own package.json too — install there as
        well:
cd frontend && npm install && cd ..

# Backend Python deps:
cd backend
python3.13 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cd ..
```

> The frontend build is invoked from the root in production (`cd frontend && ../node_modules/.bin/next build`). Locally you can run npm scripts from inside `frontend/` directly.

3. Environment variables

There are **two** env files:

- `frontend/.env.local` — for the Next.js app (and the scripts in `frontend/scripts/`).
- `backend/.env` — for the FastAPI AI Brain.

Start from the template:

```
cp frontend/.env.production.example frontend/.env.local
```

> 🔑 **Secret:** Never commit real keys. `.env.local` and `backend/.env` are git-ignored. The `*.example` file lists names only.

Frontend env (`frontend/.env.local`) — the essentials

Var	Purpose
`NEXT_PUBLIC_APP_URL`	Base URL of the app (`http://localhost:3000` locally)
`ALLOW_AUTH_BYPASS`	Dev-only: skip auth gating. **Never set in prod**
`NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public Supabase client
`SUPABASE_SERVICE_ROLE_KEY`	Server-side privileged DB access (route handlers, scripts)
`NEXT_PUBLIC_SANITY_PROJECT_ID` (`fxz10xl7`), `NEXT_PUBLIC_SANITY_DATASET` (`production`), `NEXT_PUBLIC_SANITY_API_VERSION`	Sanity client
`SANITY_API_TOKEN`	Write/mutate access for Studio + content scripts
`SANITY_WEBHOOK_SECRET`	Verify Sanity GROQ webhooks
`INGEST_SECRET`	Shared secret for the Sanity→backend ingest bridge (must match backend)
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` , `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET`	Billing
`STRIPE_PRICE_*`	One price ID per plan/interval (subscriber/student/professional × monthly/yearly)
`LOOPS_API_KEY` , `LOOPS_TEMPLATE_*`	Email
`API_KEY_HMAC_SECRET`	Hashing developer API keys
`PYTHON_BACKEND_URL` / `BACKEND_URL`	Where the Next.js app proxies AI calls (`http://localhost:8000` locally)
`NEXT_PUBLIC_SENTRY_DSN` , `SENTRY_ORG` , `SENTRY_PROJECT` , `SENTRY_AUTH_TOKEN`	Error monitoring

(Full annotated list in Doc 10 — Appendix A.)

Backend env (`backend/.env`) — from `backend/config.py`

Var	Default	Purpose
`GROQ_API_KEY`	—	Default chat LLM
`ANTHROPIC_API_KEY`	—	Alt LLM (higher roles)
`OPENAI_API_KEY`	—	**Embeddings** (required for RAG)
`GROQ_MODEL`	`llama-3.3-70b-versatile`	Subscriber model
`SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_URL`	—	DB
`SUPABASE_SERVICE_ROLE_KEY`	—	Privileged DB
`SUPABASE_JWT_SECRET`	—	Verify session JWTs from the frontend
`SUPABASE_DB_URL`	—	Direct Postgres URL (pgvector)
`SANITY_PROJECT_ID`, `SANITY_DATASET` (`production`), `SANITY_API_TOKEN`, `SANITY_API_VERSION` (`2023-10-01`)	—	Pull content for ingest
`FRONTEND_URL`	`http://localhost:3000`	CORS allow-list
`INGEST_SECRET`	—	Must equal the frontend's `INGEST_SECRET`
`SENTRY_DSN`, `ENVIRONMENT`	—	Monitoring

4. Running the frontend

```
cd frontend
npm run dev      # next dev --webpack  → http://localhost:3000
```

Other scripts (`frontend/package.json`):

Script	Does
`npm run dev`	Dev server (webpack)
`npm run build`	Production build
`npm run start`	Serve a production build
`npm run lint` / `npm run lint:strict`	ESLint (strict = 0 warnings)
`npm run typecheck`	`tsc --noEmit`
`npm run test:e2e`	Playwright e2e
`npm run bundle:check`	`scripts/check-bundle-size.sh`
`npm run smoke`	typecheck + lint + build + bundle check (the full local gate)
`npm run seed:courses`	`node scripts/seed-courses.mjs`

5. Running the backend (AI Brain)

```
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000
```

- Healthcheck: `GET http://localhost:8000/health`.
- The app initializes Sentry (if `SENTRY_DSN`), builds/loads the RAG index, builds the catalog index, then serves.
- If you don't need RAG locally, you can still run the app — chat will fail gracefully without an index, but pages and non-AI routes work.

> ⚠️ The backend talks to **production** Supabase/Sanity unless you point it at staging. Be deliberate about which dataset/DB your `.env` references before running ingest.

6. Running Sanity Studio

The Studio is mounted **inside** the Next.js app at `/studio` (route `frontend/app/studio/[[...index]]`). With `npm run dev` running, open:

```
http://localhost:3000/studio
```

You can also run the standalone Sanity dev server from `frontend/sanity/` if needed:

```
cd frontend/sanity
npx sanity dev          # standalone studio
npx sanity deploy       # deploy hosted studio (if used)
```

Config lives in `frontend/sanity/sanity.config.ts` (Studio) and `frontend/sanity/sanity.cli.ts` (CLI; `projectId: 'fxz10xl7'`, `dataset: 'production'`). Schema is in `frontend/sanity/schemaTypes/`. See Doc 03.

7. Database / Supabase

Migrations live in `supabase/migrations/` as **ordered SQL files** (`001_…` through `033_…`, plus dated ones). They are append-only history — never edit a shipped migration; add a new one.

To apply against a project (with the Supabase CLI linked):

```
supabase db push            # apply pending migrations
# or run a single file against SUPABASE_DB_URL with psql:
psql "$SUPABASE_DB_URL" -f
        supabase/migrations/033_course_chapter_ref_backfill.sql
```

`supabase/seed/` holds seed data. The Academy structure is seeded with `frontend/scripts/seed-courses.mjs` and siblings — see Doc 05 and Doc 07.

8. Verifying the full stack locally

A green end-to-end check:

1. `backend`: `uvicorn main:app --reload` → `curl localhost:8000/health` returns OK.
2. `frontend`: `npm run dev` → open `http://localhost:3000`, pages render.
3. Sign up / log in (Supabase Auth) at `/signup` / `/login`.
4. Open `/studio`, confirm you can see documents.
5. Open the AI Analyst (right sidebar or `/chat`) as a subscriber → ask a question → get a grounded, cited answer (confirms backend + RAG + Supabase JWT all wired).
6. Hit `/pricing` → start a Stripe **test-mode** checkout to confirm billing env.

9. Quality gates: lint, typecheck, tests, build

The canonical pre-push gate:

```
cd frontend && npm run smoke     # typecheck → lint → build → bundle:check
```

CI (`.github/workflows/ci.yml`) runs frontend lint + build on every push/PR to `main` . Backend deploy (`fly-deploy.yml`) triggers only on changes under `backend/**` . See Doc 08.

10. Common setup problems

Symptom	Cause	Fix
Scripts in `frontend/scripts/` can't resolve `@supabase/supabase-js`	Script run from `/tmp` or wrong cwd	Scripts **must** live and run inside `frontend/scripts/` so node resolves deps.
Lesson renders thin/legacy content	Supabase `lessons.sanity_slug` not set	Run `frontend/scripts/link-sanity-slugs.mjs` . See Doc 05.
AI Analyst returns auth error	`SUPABASE_JWT_SECRET` mismatch between frontend session and backend	Ensure backend `.env` JWT secret matches the Supabase project.
Ingest webhook 401	`INGEST_SECRET` differs between frontend and backend	Make them identical.
RAG returns nothing	No embeddings (missing `OPENAI_API_KEY`) or empty index	Set OpenAI key and run ingest (Doc 06).
Tailwind classes missing	Tailwind v4 beta PostCSS plugin not picked up	Reinstall in `frontend/` , restart dev server.
Build fails on `next build` path	Running from wrong dir	Build from `frontend/` (or via the root `vercel.json` command).

Continue to → 03 — Content Creation: Sanity CMS

03 — Content Creation: Sanity CMS

> **Verified against:** `frontend/sanity/sanity.config.ts`, `frontend/sanity/schemaTypes/*` (all 22 schema types), `frontend/app/api/webhooks/sanity/route.ts`, `frontend/lib/sanity-fetch.ts`.

Sanity is where **all editorial content** lives. This document is the working manual for editors and authors.

Table of contents

---

1. Accessing the Studio

The Studio is embedded in the Next.js app:

```
Production:  https://<app-domain>/studio
Local:       http://localhost:3000/studio
```

Route: `frontend/app/studio/[[...index]]`. Log in with your Sanity account (must be a member of project `fxz10xl7`). You can also run a standalone Studio from `frontend/sanity/` with `npx sanity dev`.

2. Project facts

Item	Value
Project ID	`fxz10xl7`
Dataset	`production`
API version	`2023-10-01` (set via `NEXT_PUBLIC_SANITY_API_VERSION`)
Studio config	`frontend/sanity/sanity.config.ts`
Desk structure	`frontend/sanity/structure.ts`
Schema types	`frontend/sanity/schemaTypes/` (registered in `index.ts`)
Write token	`SANITY_API_TOKEN` (used by scripts + ingest)
Webhook secret	`SANITY_WEBHOOK_SECRET`

3. The content model — every document type

Registered in `frontend/sanity/schemaTypes/index.ts`. Twenty-two types:

Schema (`_type`)	Studio title	Purpose / where it surfaces
`policyAnalysis`	**Analysis**	The flagship research brief. Surfaces on all six pillar sections, `/articles/[slug]`, `/read/[slug]`, search, RAG
`post`	Post	Generic article/post
`author`	Author	Byline for posts
`instructor`	Instructor	Course faculty (`/academy/faculty`)
`course`	**Course**	Academy course shell (links `academyModule` refs + instructors)
`academyModule`	Academy Module	A module within a course
`caseStudy`	**Case Study**	`/academy/case-studies`, advisory
`webinar`	Webinar / Event	`/academy/webinars`, events
`report`	**Impact Report**	`/library`, advisory reports (PDF + summary)
`definition`	Glossary Definition	`/academy/glossary`
`ticker`	System Vitals (Ticker)	The scrolling metric strip (`TickerStrip`)
`dailyInsight`	Daily Insight (Dark Strip)	The dark insight strip on home
`analystNote`	Analyst Note	AI-analyst / editorial notes
`audio`	Audio	Narration / audio blocks
`hospital`	Hospital	Hospital profiles (`/dashboard`, directory)
`rhtState`	RHT State Profile	Rural Health Transformation state pages (`/states/[state]`)
`statePerformanceIndex`	State Performance Index	The 0–100 six-pillar composite per state
`investmentDeal`	Investment Deal	`/investment-tracker` (M&A, VC, PE, IPO…)
`subscriber`	Subscriber	Newsletter/subscriber records
`category`	Category	Taxonomy
`blockContent`	(object)	Reusable Portable Text body definition

Field-level cheat sheet (most-used types)

`policyAnalysis` **(Analysis)** — `title`, `slug`, `pillar` (one of the six), `category` (subcategory, see §6), `chapterRef` (book chapter "1"–"20"), `status`, plus body (Portable Text). See §4.

`course` — `title`, `slug`, `pillar`, `type` (Format Type, e.g. cohort/self-paced), `description` (short), `meta` (e.g. "8 Weeks • Online Cohort"), `price` (e.g. "$2,995"), `instructors` (refs → `instructor`), `modules` (refs → `academyModule`, **in order**), `overview` (full syllabus, Portable Text).

`caseStudy` — `title`, `slug`, `pillar`, `chapterRef`, `clientType` (e.g. "Rural Hospital"), `summary`, `metrics` (array of strings like "40% Reduction"), `body` (Portable Text), `mainImage`.

`webinar` — `title`, `slug`, `pillar`, `chapterRef`, `description`, `date` (datetime — drives sorting), `duration`, `registrationLink` (url), `image`.

`report` (Impact Report) — `title`, subtitle, `publishedAt`, `pillar`, `chapterRef`, `accessLevel`, `coverImage`, `file` (PDF), `summary` (executive summary), `topics` (array).

`definition` — `term`, `description`, `pillars` (array, multi-pillar allowed), `chapterRef`.

`ticker` — `label` (e.g. "ER Wait Time (Avg)"), `value` (e.g. "4.2 Hours"), `trend` (e.g. "+12% YoY"), `status` (good/warning/critical/neutral → green/orange/red/blue).

`dailyInsight` — `isActive` (the app shows the most recently updated active one), `category` (QUOTE/CHART/STAT/READ/TRIVIA), `content` (headline), `link` (optional).

`investmentDeal` — `title`, `dealType` (ma/vc/pe/ipo/partnership/debt), `status` (announced/pending/closed/terminated), `announcedDate`, `closedDate`, `dealValueUsd` (in **millions**), `acquirer`, `target`, `pillar`.

`rhtState` (Rural Health Transformation state) — `stateId` (slug like `new_hampshire`), `stateName`, `awardAmount` (e.g. "$195,000,000"), `status` (Active/Pending/At Risk), `strategicFocus`, `description`, `initiatives` (array of {title, description}), `metrics` (array of {label, value, status}).

`statePerformanceIndex` — `stateId`, `stateName`, `performanceScore` (0–100 composite), `status` (Leading/Improving/Stable/At Risk), and five metric objects (`policyMetrics`, `economicsMetrics`, `technologyMetrics`, `clinicalMetrics`, `equityMetrics`) each holding 0–100 sub-scores, plus a narrative.

4. Anatomy of an Analysis (the flagship type)

The **Analysis** (`policyAnalysis`) is the workhorse. A complete Analysis has:

1. **Title** — specific and dated where relevant.
2. **Slug** — kebab-case, stable (URLs and ingest depend on it; don't churn it).
3. **Pillar** — exactly one of: Policy, Economics, Technology, Clinical, Equity, Operations.
4. **Category** — a pillar-scoped subcategory (the dropdown lists all valid pairs; see §6).
5. **chapterRef** — optional book chapter number "1"–"20" tying the brief to *Transforming American Healthcare*.
6. **status** — editorial state.
7. **Body** — Portable Text: headings, paragraphs, callouts, stats, sources/citations.

> **Quality bar:** every factual/statistical claim in an Analysis body must be verifiable and sourced. The repo's content-correction history (`CONTENT_CORRECTIONS.md` , `ANALYSIS_CONTENT_STANDARDS.md`) records prior removals of fabricated stats. When in doubt, cut the claim or cite it. See §10.

5. Portable Text & block content

Bodies use Sanity **Portable Text** via the shared `blockContent` type (`frontend/sanity/schemaTypes/blockContent.ts`). It is rendered with `frontend/sanity/schemaTypes/PortableTextComponents.tsx` on the web side. Supported blocks typically include headings, rich text, lists, links, images, code (`@sanity/code-input`), and custom callouts/audio (`AudioPlayer.tsx` , `audio` type).

When content is fetched for the Academy, the Portable Text body is packed into the renderer; `frontend/components/AcademyContent.tsx` is the **gold-standard renderer** — match its block structure when authoring programmatically.

6. Pillars, categories, and book-chapter linkage

Pillar → Category pairs (the valid set, from the `policyAnalysis` schema):

- **Policy:** Regulation & Legislation · Public Health Mandates · Global & Comparative Policy · Policy Feasibility Studies
- **Economics:** Value-Based Care Models · Market & Finance · Labor & Workforce Strategy · Healthcare Investment Trends

- **Technology:** AI & Machine Learning · Digital Health & Telemedicine · Data Security & Governance · Tech-Enabled Workflow
- **Clinical:** Hospital-at-Home · Precision Medicine · Virtual Care Models · Population Health
- **Equity:** SDOH Integration · Algorithmic Bias · Access Disparity · Community Engagement
- **Operations:** (Revenue Cycle, Supply Chain, Workforce, Compliance, Payer Network — surfaced under `/operations/*`)

Book chapter (`chapterRef`) links content to the companion book. Many types carry it (`policyAnalysis`, `caseStudy`, `webinar`, `report`, `definition`). It powers "From the Book" cross-links (`FromTheBook.tsx`).

7. Publishing workflow & what happens on publish

1. **Draft** in Studio → fill all required fields → **Publish**.
2. The published document becomes visible to the live site via GROQ queries (`frontend/lib/sanity-fetch.ts`). Next.js may serve cached/ISR data — there's a daily revalidate cron (`vercel.json` → `/api/cron/revalidate`); for an instant refresh, trigger a revalidation or redeploy.
3. **If** the type is in the ingest filter (see §8), publishing fires the GROQ webhook → backend ingest → RAG index updates. This is how new Analyses become answerable by the AI Analyst.

8. The Sanity → RAG ingest webhook

Defined by `frontend/app/api/webhooks/sanity/route.ts`. It receives Sanity's GROQ webhook and forwards it to the backend `POST /api/ingest/webhook` with `Authorization: Bearer <INGEST_SECRET>` and `X-Sanity-Signature`.

Configure the webhook in Sanity (manage.sanity.io → API → Webhooks):

Setting	Value
URL	`https://<app-domain>/api/webhooks/sanity`
Secret	same value as `INGEST_SECRET` (and `SANITY_WEBHOOK_SECRET`)
Trigger	on create/update/delete
Filter (`_type in [...]`)	`policyAnalysis`, `post`, `academyModule`, `caseStudy`, `definition`, `analystNote`, `webinar`, `report`

> Types **not** in this filter (e.g. `ticker`, `dailyInsight`, `investmentDeal`, `statePerformanceIndex`) are **display-only** — they render on pages but are not ingested into RAG. That's intentional.

If RAG falls behind Sanity, re-run a full ingest from the backend (see Doc 06 §6).

9. Programmatic content creation (scripts & AI generation)

Beyond the Studio UI, content is created and maintained at scale by scripts in `frontend/scripts/` and generation prompts in `frontend/sanity/`.

Generation prompts (`frontend/sanity/`): `Prompt_template_Claude.txt`, `Prompt_template_Claude_v2.txt`, `Prompt_Template_Final.txt`, `Prompt_template_Academy_v1.txt`, `ultimate_prompt*.txt`, `master_instructions_block.txt`. These encode the editorial voice and structure for AI-assisted drafting. `generate_sanity_content.py` drives generation; JSON outputs land in `frontend/sanity/content/*.json` and are imported with the bulk-import scripts.

Content seed/import scripts (run from `frontend/scripts/`):

- `bulk_import.js`, `import.js`, `import_one.js` — push JSON documents into Sanity.
- `import-glossary.js`, `seed-webinars.js`, `seed-reports.js`, `seed-caseStudies.js`, `seed-ticker.js` — type-specific seeds.
- `seed-sanity-performance-index.ts`, `seed-sanity-rht.ts` — state index + RHT profiles.

Content maintenance scripts (Analysis quality program): `audit-analysis-length.mjs`, `clean-policy-analysis.mjs`, `neutralize-unverifiable.mjs`, `purge-htr-claims.mjs`, `scan-htr-claims.mjs`, the `expand-batch-*.mjs` family, and the `triage-*` scripts. Full reference in Doc 07.

> ⚠️ All content scripts use `SANITY_API_TOKEN` and write to the **production** dataset. Always dry-run/inspect target IDs first. Several scripts (e.g. `triage-delete.mjs`, `clean-garbage.mjs`) delete documents.

10. Editorial standards & quality bar

The repo ships content standards you must follow when authoring Analyses:

- `ANALYSIS_CONTENT_STANDARDS.md` — structure + sourcing rules for Analyses.

- `CONTENT_CORRECTIONS.md` / `CONTENT_PROMPT.md` / `CONTENT_PROMPT_EDITORIAL.md` — the editorial voice and the log of corrections.
- **Hard rules** distilled from project history:
  - Every statistic must be web-verifiable and sourced; unverifiable claims are removed or neutralized.
  - Do not fabricate "HTR" proprietary numbers — those were purged.
  - Keep slugs stable once published.
  - Match the gold-standard renderer's block structure.

11. Backups

Sanity exports are kept in two places at repo root/Studio:

- `sanity-backups/` (repo root) — dataset export snapshots.
- `frontend/sanity/my-backup.tar.gz` — a Studio tarball.

To take a fresh export:

```
cd frontend/sanity
npx sanity dataset export production ../../sanity-
        backups/production-$(date +%Y%m%d).tar.gz
```

To restore (⚠️ destructive to the target dataset):

```
npx sanity dataset import <backup>.tar.gz production --replace
```

Continue to → 04 — Content & Data: Supabase

04 — Content & Data: Supabase

> **Verified against:** `supabase/migrations/001…033` + dated migrations, `backend/services/db.py`, `frontend/lib/supabase.ts`, `frontend/lib/auth.ts`.

Supabase is the **application database, auth provider, file storage, and vector store**. Sanity holds prose; Supabase holds *people, permissions, progress, money, and embeddings.*

Table of contents

---

1. What Supabase provides

Capability	Used for
Postgres	All relational app data (tables below)
Auth (GoTrue)	Email/password + session JWTs (`auth.users`)
Storage	Certificate PDFs, audio uploads, report files
pgvector	RAG semantic search (HNSW index)
RLS	Per-user data isolation enforced in the DB

Clients:

- Frontend server/client: `frontend/lib/supabase.ts` (`@supabase/ssr`), anon key for the browser, service-role key for privileged server work.
- Backend: `backend/services/db.py` — a service-role Supabase singleton; also a direct Postgres URL (`SUPABASE_DB_URL`) for pgvector.

2. Migrations: the source of schema truth

`supabase/migrations/` contains **34 ordered, append-only** SQL files. The schema is whatever these produce when run in order.

#	Migration	Adds
001	profiles_and_roles	`profiles`, `user_roles` (enum `user_role`), `subscriptions` (enum `subscription_status`), `set_updated_at()` trigger
002	content_data	hospitals, state metrics, benchmarks, time-series
003	academy	`course_enrollments`, `module_progress`, `certifications`
004	advisory	advisory clients & reports
005	rag_vectors	pgvector store (`rag_documents`)
006	rls_policies	base RLS
007	hybrid_search	hybrid (vector+keyword) search SQL function
008	rls_audit	RLS audit
009	referrals	`referral_codes`, `referral_events`
010	community	`community_*` tables
011	api_keys	developer API keys
012	survey	survey editions/responses
013	role_audit	role change auditing
014	webhook_inbox	inbound webhook log
015	rag_query_log	log of RAG queries
016	bookmarks	`bookmarks`
017	ticker_cache	cached ticker metrics
018	role_change_log	role change log
019	user_learning_paths	personalized learning paths
020	rag_query_log_pruning	retention
021	pgvector_hnsw_maintenance	HNSW index maintenance
022	hti_scores	Health Tech Index scores
023	api_key_rotation_and_directory	key rotation + member directory
024	rag_feedback	thumbs up/down on RAG answers
025	wire_comments	The Wire comments
026	chapter_notes	book chapter notes
027	digest_opt_in	email digest preference
028	course_schema	**new course player:** `courses`, `tracks`, `lessons`, `quizzes`,

#	Migration	Adds
		`quiz_questions`, `quiz_options`, `audio_slots`, progress + attempts tables
029	course_pillar_level	pillar/level on courses
030	course_featured	featured flag
031	lesson_sanity_slug	`lessons.sanity_slug` (link to Sanity body)
032	course_chapter_ref	course↔book chapter
033	course_chapter_ref_backfill	backfill above
2026-04-11	beta_access_codes	`beta_access_codes`

> **Rule:** never edit a migration that has shipped. To change schema, add the next-numbered file. Apply with `supabase db push` or `psql "$SUPABASE_DB_URL" -f <file>`.

3. Identity, roles & subscriptions

`profiles` — one row per `auth.users` id: `email`, `full_name`, `avatar_url`, `org_name`, `bio`, timestamps. Auto-`updated_at` via trigger.

`user_roles` — `role` (enum `user_role`, default `free`), `expires_at` (NULL = permanent), `granted_by` (admin who granted). The role ladder: `free` → `subscriber` → `professional` → `advisory` → `admin`. Role changes are logged to `role_change_log` (auditing in `/admin/role-changes`).

`subscriptions` — one row per user: `stripe_customer_id`, `stripe_subscription_id`, `plan`, `status` (enum), `current_period_*`, `cancel_at_period_end`. This **mirrors Stripe** — the Stripe webhook (`/api/stripe/webhook`) keeps it in sync. `stripe_customers` and `stripe_events` track the raw Stripe side; `stripe_events` provides idempotency.

> **Source of truth for billing is Stripe.** Supabase is a fast-read mirror. Never edit `subscriptions` by hand to grant access — either change Stripe or grant a role in `user_roles`.

4. The Academy data tables

There are **two generations** of Academy schema, both present:

- **Legacy/CMS-driven (mig. 003):** `course_enrollments`, `module_progress`, `certifications` — keyed to Sanity slugs (`course_slug`, `module_slug`).
- **Course Player (mig. 028+):** a fully relational model:
  - `courses` → `tracks` → `lessons` (FK chain, cascade delete).
  - `lessons` columns: `track_id`, `pillar` (`htr_pillar` enum), `order`, `slug`, `title`, `summary`, `estimated_minutes`, `objectives` (JSONB), `content_blocks` (JSONB — *legacy thin content*), `tags`, `related_lesson_ids`, `is_published`, and **`sanity_slug`** (mig. 031 — links to the rich Sanity body).
  - `quizzes` → `quiz_questions` → `quiz_options`.
  - `audio_slots`, `learner_audio_uploads` — narration & learner audio.
  - Progress/attempts: `course_player_enrollments`, `course_lesson_progress`, `course_quiz_attempts`.
  - `lesson_bookmarks`, `lesson_notes`.

> **The `sanity_slug` rule (critical):** if a lesson's `sanity_slug` is empty, the app renders the thin `content_blocks` JSON instead of the rich Sanity body. After posting lesson content to Sanity, set each lesson row's `sanity_slug` = the Sanity slug/`_id` using `frontend/scripts/link-sanity-slugs.mjs`. Full Academy mechanics in Doc 05.

`certifications` — `cert_name`, `cert_type` (completion/professional/cme), `course_slug`, `issued_at`, `expires_at`, `cert_url` (Storage PDF), `verification_hash` (powers the public `/verify/[hash]` page), `metadata`.

5. RAG / pgvector tables

- `rag_documents` — chunked, embedded content (vector column + metadata: source slug, type, pillar). HNSW index maintained by migration 021.
- `rag_query_log` — every RAG query (pruned by 020).
- `rag_feedback` — thumbs up/down per answer (024).
- Hybrid search SQL function (007) combines vector similarity with keyword match; the backend `HybridRetriever` calls it. See Doc 06.

6. Community, bookmarks, notes, referrals

Feature	Tables	Surfaces at
Community forums	`community_categories`, `community_threads`, `community_posts`, `community_upvotes`	`/community`, `/connect/forums`
The Wire comments	`wire_comments`, `wire_comment_upvotes`	`/the-wire`
Bookmarks	`bookmarks`, `lesson_bookmarks`	`/saved`, `/account/bookmarks`
Notes	`lesson_notes`, `chapter_notes`	lessons, book reader
Referrals	`referral_codes`, `referral_events`	`/account/referrals`
Survey	`survey_editions`, `survey_responses`	`/survey`
Developer API	`api_keys` (HMAC-hashed, rotatable)	`/account/api-keys`, `/api/v1/*`
Digest	`digest_opt_in` column	email digest cron
Beta gating	`beta_access_codes`	`/beta`, `/api/beta/verify`

7. Row-Level Security (RLS)

RLS is **on** for user-owned tables (migrations 006, 008). The pattern:

- A user may `select` / `insert` / `update` only rows where `user_id = auth.uid()`.
- Service-role (backend, server route handlers) bypasses RLS — so privileged operations run server-side with `SUPABASE_SERVICE_ROLE_KEY`, never from the browser.
- Admin reads (e.g. `/admin/users`) go through server route handlers using the service role + an in-app role check.

> ⚠️ Never expose `SUPABASE_SERVICE_ROLE_KEY` to the client. It is server-only.

8. Storage buckets

Supabase Storage holds binary artifacts:

- Certificate PDFs (`certifications.cert_url`).
- Learner audio uploads (`learner_audio_uploads`).
- Report files where not stored in Sanity.

9. Common admin operations

Grant a role to a user (e.g. comp a subscription) — preferred over editing `subscriptions`:

```
INSERT INTO public.user_roles (user_id, role, granted_by)
VALUES ('<user-uuid>', 'professional', '<admin-uuid>')
ON CONFLICT (user_id) DO UPDATE SET role = EXCLUDED.role,
        granted_at = NOW();
-- (also logged via role_change_log; prefer the /admin/users UI
        which logs automatically)
```

Look up a user's effective access:

```
SELECT p.email, r.role, r.expires_at, s.plan, s.status,
        s.current_period_end
FROM profiles p
LEFT JOIN user_roles r   ON r.user_id = p.id
LEFT JOIN subscriptions s ON s.user_id = p.id
WHERE p.email = '<email>';
```

Issue/verify a beta code: insert into `beta_access_codes`; users redeem at `/beta`.

Prefer the **Admin UI** (`/admin/users`, `/admin/access-codes`, `/admin/role-changes`) — it writes audit rows for you.

10. Seeding & data loaders

Goal	Script
Seed Academy courses (structure)	`frontend/scripts/seed-courses.mjs` (also `npm run seed:courses`), `seed-all-courses.mjs`, `seed-courses-tier2.mjs`
Link lesson rows to Sanity bodies	`frontend/scripts/link-sanity-slugs.mjs`
Load CMS hospital data	`backend/scripts/load_cms_hospitals.py`, `sync_hospitals.py`
Load CMS quality scores / HTI	`backend/scripts/load_cms_quality_scores.py`, `sync_hti_scores.py`
Seed Supabase content	`frontend/scripts/seed-supabase.js`, `seed-content.ts`
Reset DB (⚠️ destructive)	`frontend/scripts/reset-database.js`

> ⚠️ **Seed scripts are upsert-only (no delete) by design.** Course membership lives in Supabase. To *remove* a lesson from a course you must delete the DB row directly (and check all tracks for dupes/orphans) — re-running a seed will not remove it. See Doc 05.

Continue to → 05 — The Academy System

05 — The Academy System

> **Verified against:** `supabase/migrations/028…033`, `frontend/lib/course-api.ts`, `frontend/components/AcademyContent.tsx`, `frontend/app/academy/*`, `frontend/app/api/academy/certificates/route.ts`, `CONTENT_TEMPLATE.py`, `frontend/scripts/seed-courses.mjs`, `frontend/scripts/link-sanity-slugs.mjs`.

The Academy is the learning side of the platform: courses, tracks, lessons, quizzes, progress tracking, certificates, and personalized learning paths. It is the most intricate subsystem because it spans **Supabase (structure + progress)** and **Sanity (rich lesson bodies)** simultaneously.

Table of contents

---

1. The hybrid data model

```
SUPABASE                                        SANITY
─────────                                       ──────

courses ──┐
          ├─< tracks ──< lessons ──┐            academyModule /
policyAnalysis
          │                        │  link  (the rich Portable
Text BODY)
          │     lessons.sanity_slug├─────────►  document.slug /
_id
          │                        │
          └─< quizzes ──< quiz_questions ──< quiz_options

course_player_enrollments      ← who's enrolled
course_lesson_progress         ← which lessons completed
course_quiz_attempts           ← quiz scores
certifications                 ← issued certs (PDF in Storage)
```

The golden rule: a `lesson` row in Supabase carries metadata (title, order, pillar, objectives) and a `sanity_slug`. The lesson's *displayed content* comes from the Sanity document whose slug equals `sanity_slug`. If `sanity_slug` is blank, the renderer falls back to the thin legacy `content_blocks` JSON on the row.

2. Academy routes

Route	Purpose
`/academy`	Academy home
`/academy/courses`	Course catalog
`/academy/tracks`	Learning tracks
`/academy/modules`	Modules
`/academy/case-studies`	Case studies (Sanity `caseStudy`)
`/academy/faculty`	Instructors (Sanity `instructor`)
`/academy/glossary`	Definitions (Sanity `definition`)
`/academy/webinars`	Events (Sanity `webinar`)
`/academy/getting-started`	Onboarding
`/academy/personalized-learning`	Standalone personalized path page (not a tab)
`/academy/medicaid`	Medicaid course area
`/account/courses`	A learner's enrolled courses + progress
`/verify/[hash]`	Public certificate verification

3. The course-api layer

`frontend/lib/course-api.ts` is the typed gateway to the course player. Key functions:

Function	Does
`getCourseWithProgress(...)`	Loads a course + its tracks/lessons + the caller's progress
`enrollUser(...)`	Creates a `course_player_enrollments` row
`updateLessonProgress(...)`	Marks lesson complete / records time
`recordQuizAttempt(...)`	Writes a `course_quiz_attempts` row
`updateAudioSlot(...)`	Updates narration audio for a lesson slot
`searchLessons(query, limit)`	Full-text lesson search
`getCoursesByPillar(pillar)`	Catalog filtered by pillar
`getCoursesByChapter()`	Courses grouped by book chapter
`seedCourseFromJson(courseData)`	Programmatic course seeding

The rendering of a lesson body is done by `frontend/components/AcademyContent.tsx` — the *gold-standard renderer*. Any programmatic content you create must produce blocks this component understands.

4. Authoring a lesson — the canonical workflow

This is the proven, repeatable process. **Do one lesson fully, end-to-end, before starting the next** — never batch.

1. **Write the lesson body** as a Python script that imports the block helpers:

   ```
   exec(open('CONTENT_TEMPLATE.py').read())   # ALWAYS start
           with this line
   # ...build blocks with blk(), h2(), callout(), stat_grid(),
           etc...
   ```

   Never reimplement block helpers inline — always `exec` the template.
2. **Web-verify every claim** in the lesson. Cite sources. No fabricated statistics.
3. **POST the body to Sanity** (the script pushes a document with a stable slug).
4. **Set the Supabase link:** update the lesson row's `sanity_slug` = the Sanity slug via `frontend/scripts/link-sanity-slugs.mjs`. Without this, the app shows thin legacy content.
5. **Verify in the app:** open the course player, confirm the rich body renders via `AcademyContent.tsx`.

6. Repeat for the next lesson.

> **Why one-at-a-time:** batching wastes credits reconstructing state after session limits, and makes it hard to verify each lesson. This is a hard project rule.

5. The CONTENT_TEMPLATE block system

`CONTENT_TEMPLATE.py` defines the **only** block types the renderer supports. Inventing new block shapes breaks rendering. The vocabulary:

Text & structure

- `blk()` — paragraph.
- `h2()` — section header (auto-numbered "SECTION #1…", indigo separator).
- `h3()` — sub-section header (small caps, indigo bullet).
- `callout()` — 💡 KEY CONCEPT card.
- `highlight()` — 📌 REMEMBER THIS (amber).
- `quote()` — pullquote (dark slate card).

Visual learning

- `analogy()` — 🔗 ANALOGY (violet). **Max once per lesson.**
- `stat_grid()` — 📊 2–4 stat cards (big number + label + context). **Max 4.**
- `example()` — 🌍 REAL-WORLD EXAMPLE (teal). **Named real orgs/cases only.**
- `steps()` — 📋 numbered process steps.
- `compare()` — ⚖️ two-column comparison (rose vs emerald).
- `warning()` — ⚠️ caveats / common mistakes.

Media — image/audio/video blocks (see the rest of `CONTENT_TEMPLATE.py`).

There is also an **editorial** variant `CONTENT_TEMPLATE_EDITORIAL.py` + `CONTENT_PROMPT_EDITORIAL.md` for editorial (non-lesson) pieces.

6. Quizzes

Schema (mig. 028): `quizzes` → `quiz_questions` → `quiz_options`. A quiz attaches to a course/lesson; questions have ordered options with a correct flag. Learner attempts land in `course_quiz_attempts` (score, timestamp). The quiz format is fixed — match existing seeded quizzes when adding new ones.

7. Progress, enrollment & certificates

- **Enroll:** `enrollUser` → `course_player_enrollments`.
- **Progress:** `updateLessonProgress` → `course_lesson_progress`; course completion % rolls up.
- **Certificate:** `POST /api/academy/certificates` issues a certificate. It is **idempotent** (returns the existing one if already issued), writes a `certifications` row with a `verification_hash`, stores the PDF URL in `cert_url`, and emails the learner via **Loops** (`sendEvent`). The certificate is publicly verifiable at `/verify/[hash]`.

8. Personalized Learning

- Frontend: standalone page `/academy/personalized-learning` (it is a **page, not a tab**).
- API: `/api/personalized-learning` (+ `/audio`) → backend router `backend/routers/personalized_learning.py`.
- Data: `user_learning_paths` (mig. 019). The backend tailors a path from the user's role/interests and can generate narration audio.

9. Seeding courses & linking Sanity bodies

Step	Command (run from `frontend/`)
Seed course structure	`node scripts/seed-courses.mjs` (or `npm run seed:courses`)
Seed tier-2 / all courses	`node scripts/seed-courses-tier2.mjs`, `node scripts/seed-all-courses.mjs`
Link lesson rows → Sanity bodies	`node scripts/link-sanity-slugs.mjs`
Audit course integrity	`node scripts/audit-courses.mjs`
Remove redundant modules	`node scripts/delete-redundant-academy-modules.mjs`

> 🔑 These scripts read `SUPABASE_SERVICE_ROLE_KEY` and `SANITY_API_TOKEN` from `frontend/.env.local`, and **must run from inside `frontend/scripts/`** so node resolves `@supabase/supabase-js`.

10. Maintenance gotchas (read before touching)

1. **Seed scripts are upsert-only — they never delete.** To remove a lesson from a course, delete the **DB row** directly, then check *all* tracks for duplicates/orphans. Re-seeding will not clean up.
2. `sanity_slug` **must be set** after posting content, or the app renders thin legacy `content_blocks`. Run `link-sanity-slugs.mjs`.
3. The linking `.mjs` **must live in** `frontend/scripts/` (not `/tmp`) or node can't resolve dependencies.
4. **Never batch lesson authoring.** One lesson fully (write → verify → POST → link → confirm) before the next.
5. **Always** start lesson scripts with `exec(open('CONTENT_TEMPLATE.py').read())`. Don't reinvent block helpers.
6. Course membership lives in **Supabase**, not Sanity — deleting a Sanity doc doesn't unenroll anyone or remove the lesson row.

Continue to → 06 — The AI Analyst & RAG Backend

06 — The AI Analyst & RAG Backend

> **Verified against:** `backend/main.py`, `backend/config.py`, `backend/services/{llm,retrieval,indexing,auth,catalog_search,tools}.py`, `backend/routers/{chat,ingest,api_v1,vermont_ops,personalized_learning}.py`, `frontend/app/api/{chat,suggest,webhooks/sanity}/route.ts`.

The **AI Analyst** is the platform's conversational research assistant. It is a retrieval-augmented generation (RAG) system backed by the FastAPI "HTR AI Brain" (v4.2.0). This document explains how it works and how to operate it.

Table of contents

---

1. Where the AI shows up

Surface	Component / route
Right-sidebar widget (every page)	`frontend/components/RightSidebar.tsx`
Full-page chat	`/chat`
Follow-up suggestions	`/api/suggest`
Personalized learning generation	`/academy/personalized-learning` → `/api/personalized-learning`

The frontend never calls the LLM directly. It calls Next.js route handlers (`/api/chat`, `/api/suggest`) which authenticate the Supabase session and proxy to the backend (`BACKEND_URL`).

2. The request path end-to-end

```
Browser (RightSidebar / /chat)
   │  POST /api/chat   (Next.js route handler)
   ▼
frontend/app/api/chat/route.ts
   │  - validates Supabase session, attaches JWT
   │  - fetch() → BACKEND_URL  POST /api/chat
   ▼
backend/routers/chat.py  (require_subscriber)
   │  1. verify JWT (services/auth.py, SUPABASE_JWT_SECRET)
   │  2. resolve role  →  pick LLM
(services/llm.get_llm_for_role)
   │  3. retrieve context (HybridRetriever + rerank + Vermont
boost)
   │  4. (professional/advisory/admin) attach ReAct tools
   │  5. stream answer + citations
   ▼
StreamingResponse  →  relayed to browser token-by-token
```

CORS is locked to `FRONTEND_URL` + `localhost:3000`. Rate limiting via `slowapi`. Sentry wraps everything.

3. Model routing by role

`backend/services/llm.py → get_llm_for_role(role)`. Every model is wrapped in a `FallbackLLM` so a provider outage degrades gracefully rather than failing.

Role	Primary model	Fallback chain
`free` / `student`	Groq `llama-3.1-8b-instant` (`MODEL_FREE`)	→ OpenAI `gpt-4o-mini`
`subscriber` / `professional`	Groq `llama-3.3-70b-versatile` (`MODEL_SUBSCRIBER`)	→ `llama-3.1-8b-instant` → `gpt-4o-mini`
`advisory` / `admin`	Anthropic `claude-sonnet-4-6` (`MODEL_ADVISORY`)	→ `llama-3.3-70b-versatile` → `gpt-4o-mini`

- **Embeddings:** OpenAI (`EMBEDDING_MODEL`) — required for RAG. Without `OPENAI_API_KEY`, ingest and retrieval cannot embed.
- Models are configurable via env (`GROQ_MODEL`, etc.). Anthropic is an **optional** dependency — if the package/key is absent, advisory/admin

gracefully fall back to the subscriber model.

4. Retrieval pipeline

In `backend/services/retrieval.py` and `indexing.py`:

1. **Index build/load** (`build_index` / `load_index`) — pulls content (`fetch_sanity_content`) and the pgvector store.
2. `HybridRetriever` — combines vector similarity (pgvector, hybrid search SQL fn from migration 007) with keyword matching. `StaticNodeRetriever` is used where a fixed node set is appropriate.
3. **Rerank** — `rerank_nodes` via FlashRank (`get_ranker`) reorders candidates by relevance. (`MetadataReplacementNodePostprocessor` swaps in full-window context.)
4. **Vermont boost** — `boost_vermont_nodes` raises Vermont-specific results because Vermont is the flagship use-case.
5. **Citations** — `extract_citations` attaches source references so the UI can show what grounded each answer.

Tuning levers live in `config.py` (e.g. `MAX_SYSTEM_PROMPT_LEN`) and the retriever constructor (top-k, score thresholds).

5. The agentic (ReAct) pipeline & tools

For `professional`, `advisory`, `admin` roles (`AGENTIC_ROLES` in `chat.py`), the chat uses a LlamaIndex **ReActAgent** with function tools instead of plain context-stuffing. The agent can call tools, observe results, and reason in steps.

`backend/services/tools.py → ALL_TOOLS`:

Tool	Function name	What it answers
State metrics	`query_state_metrics`	Per-state six-pillar performance data
Research Lab	`list_research_lab_tools`	Which interactive Lab tools exist
VT hospital financials	`query_vermont_hospital_financials`	Vermont hospital financial data
VT bed capacity	`query_vermont_bed_capacity`	Current bed capacity
Act 167 recommendations	`query_act167_recommendations`	Vermont Act 167 guidance
Best transfer	`find_best_transfer`	Optimal patient transfer destination
VT HSA population	`query_vermont_hsa_population`	Health Service Area population
VT system summary	`query_vermont_system_summary`	System-wide Vermont snapshot

Lower roles (subscriber) get RAG-only `ContextChatEngine` (no tools).

6. Ingest: keeping RAG in sync with Sanity

`backend/routers/ingest.py` exposes:

Endpoint	Auth	Purpose
`POST /api/ingest` (202)	`INGEST_SECRET`	Kick off a full/background re-index job
`GET /api/ingest/status`	`INGEST_SECRET`	Poll job status
`POST /api/ingest/webhook` (202)	Bearer `INGEST_SECRET`	Incremental ingest of a single changed doc (called by `/api/webhooks/sanity`)

Incremental flow (normal operation): Editor publishes in Sanity → Sanity GROQ webhook → Next.js `/api/webhooks/sanity` → backend `/api/ingest/webhook` → `_run_incremental(doc_id, index)` parses, chunks, embeds, upserts into pgvector. The new content is answerable within seconds.

Full re-index (recovery): if RAG drifts (e.g. after a bulk content migration or restore), POST `/api/ingest` to rebuild. Poll `/api/ingest/status`.

> The ingest filter (which `_type`s reach RAG) is set on the Sanity webhook: `policyAnalysis`, `post`, `academyModule`, `caseStudy`, `definition`, `analystNote`, `webinar`, `report`. See Doc 03 §8.

7. Suggestions & the feature catalog

- **`/api/suggest`** (open, no subscription) generates follow-up questions.
- **`services/catalog_search.py`** holds a semantic index of the platform's own pages/tools (`platform_catalog.py → HTR_TOOLS_CATALOG_TEXT`). When a user's question maps to a feature (e.g. "compare states"), the AI surfaces a link to the right page (`find_relevant_tools_semantic`, `build_guaranteed_lab_section`). This is what lets the Analyst say "use the State Comparison tool at /compare-states."

8. The developer API (`/api/v1`)

`backend/routers/api_v1.py`. Authenticated by **API key** (header `X-HTR-API-Key`), validated by `require_api_key` against the `api_keys` table (HMAC-hashed, rotatable — migration 023). Endpoints:

Endpoint	Returns
`GET /api/v1/states`	All state performance profiles
`GET /api/v1/states/{state_id}`	One state profile
`GET /api/v1/survey/results`	Aggregated survey results (current edition)

Users manage keys at `/account/api-keys` (`/api/keys/create`, `/revoke`, `/rotate`).

9. Vermont Ops tools

`backend/routers/vermont_ops.py` powers the operational Vermont features (subscriber+). Surfaced in the app at `/bed-capacity`, `/connect/alerts`, etc.

Endpoint	Purpose
`GET /vermont/bed-capacity`	Current bed capacity
`PATCH /vermont/bed-capacity/{hospital_id}`	Update capacity
`GET /vermont/alerts`	Capacity alerts
`POST /vermont/transfer-log`	Log a patient transfer
`GET /vermont/transfer-log`	Read transfer log

These also back the agentic tools in §5 (e.g. `find_best_transfer`).

10. Operating, tuning & troubleshooting

Health & monitoring

- `GET /health` — liveness (Fly + Railway healthcheck path).
- Sentry captures backend exceptions (`SENTRY_DSN`, `traces_sample_rate=0.1`).
- `rag_query_log` records queries; `rag_feedback` records 👍/👎. Review these to spot weak answers.

Common issues

Symptom	Likely cause	Fix
401 from chat	JWT secret mismatch	Backend `SUPABASE_JWT_SECRET` must match Supabase project
"No context"/empty answers	Empty/stale RAG index	Run full ingest `POST /api/ingest`
Ingest webhook ignored	`INGEST_SECRET` mismatch or doc `_type` not in filter	Align secrets; check webhook filter
Slow first response	Backend cold start (Fly scale-to-zero, `min_machines_running=0`)	Expected; first call warms it. Raise min machines if needed
Advisory role gets generic answers	Anthropic key/package missing → fell back to Groq	Set `ANTHROPIC_API_KEY`, install the Anthropic LlamaIndex integration
Embeddings error	Missing `OPENAI_API_KEY`	Set it; embeddings are OpenAI-only
Rate-limited	`slowapi` limits hit	Tune limits in `main.py`

Cost control: Groq is the cheap default; Anthropic only fires for advisory/admin. Embeddings (OpenAI) cost scales with ingest volume — full re-indexes are the expensive operation, so prefer incremental webhook ingest.

Continue to → 07 — Tooling & Scripts Reference

07 — Tooling & Scripts Reference

> **Verified against:** `frontend/package.json`, `frontend/scripts/` (~90 files), `backend/scripts/`, repo-root `scripts/`, `scripts/generate-narration-piper.sh`, `frontend/scripts/check-bundle-size.sh`, generator helpers (`generate_pptx.py`, `generate_whitepaper_docx.py`, `scripts/legacy-python/`).

This is the field manual for every command and script. Scripts fall into five families: **build/QA**, **content (Sanity)**, **data/Academy (Supabase)**, **media/narration**, and **document generation**.

Table of contents

---

1. Safety rules for all scripts

> ⚠️ Read these before running anything that writes.

1. **Content scripts target the PRODUCTION Sanity dataset** (`production`) and **Supabase** via service-role keys. There is no staging guard. Inspect target IDs first.
2. **Run from `frontend/scripts/`** so node resolves `@supabase/supabase-js` and other deps. A script copied to `/tmp` will fail.
3. **Seed scripts are upsert-only** — they never delete. Removing content requires a direct DB delete + orphan check.

4. **Several scripts delete documents** (`triage-delete.mjs`, `clean-garbage.mjs`, `delete-redundant-academy-modules.mjs`, `reset-database.js`). Treat as destructive.
5. Scripts read secrets from `frontend/.env.local`. Confirm it points at the intended project.

2. Build & QA commands

Run from `frontend/`:

Command	Purpose
`npm run dev`	Dev server (`next dev --webpack`)
`npm run build`	Production build
`npm run start`	Serve production build
`npm run lint`	ESLint
`npm run lint:strict`	ESLint, fail on any warning
`npm run typecheck`	`tsc --noEmit`
`npm run test:e2e`	Playwright e2e (`frontend/e2e/`)
`npm run test:e2e:install`	Install Playwright chromium
`npm run bundle:check`	`scripts/check-bundle-size.sh` — fails if largest non-`/studio` chunk exceeds threshold (Studio is excluded; it's ~4 MB by design)
`npm run smoke`	typecheck → lint → build → bundle:check (**the pre-push gate**)
`npm run seed:courses`	Seed Academy courses

3. Content scripts (Sanity)

Import / seed (run from `frontend/`, `node scripts/<name>`):

Script	Purpose
`bulk_import.js`, `import.js`, `import_one.js`	Push JSON documents into Sanity
`import-glossary.js`	Seed glossary `definition` docs
`seed-webinars.js`	Seed `webinar` docs
`seed-reports.js`	Seed `report` docs
`seed-caseStudies.js`	Seed `caseStudy` docs
`seed-ticker.js`	Seed `ticker` metrics
`seed-sanity-performance-index.ts`	Seed `statePerformanceIndex`
`seed-sanity-rht.ts`	Seed `rhtState` profiles
`convert_sanity_to_supabase.mjs`	Convert Sanity content into Supabase rows

Generation: `frontend/sanity/generate_sanity_content.py` + the prompt templates (`Prompt_Template_Final.txt`, `ultimate_prompt*.txt`, `master_instructions_block.txt`) drive AI-assisted drafting; JSON outputs land in `frontend/sanity/content/*.json`, then are imported.

4. Content quality / audit scripts

The Analysis-quality program (used to deep-rewrite and source all 77 Analyses):

Script	Purpose
`audit-analysis-length.mjs`	Flag too-short/thin Analyses
`clean-policy-analysis.mjs`	Clean/normalize Analysis bodies
`neutralize-unverifiable.mjs`	Remove/soften unverifiable claims
`scan-htr-claims.mjs` / `purge-htr-claims.mjs`	Find & remove fabricated proprietary "HTR" stats
`extract-low51-claims.mjs`, `sweep-low51.mjs`, `scan-remaining79.mjs`	Targeted claim sweeps
`expand-batch-{clinical,econ,equity,policy,tech}-N.mjs`, `expand-batch-final.mjs`	Batch-expand thin briefs by pillar
`fix-broken-sanity-links.mjs`, `remap-interop-slugs.mjs`, `backfill-analysis-chapterref.mjs`, `backfill-editorial-pillar-chapter.mjs`	Link/metadata repair
`triage-queue.mjs`, `triage-delete.mjs`, `clean-garbage.mjs`	Triage & delete junk docs (⚠️ destructive)
`append-vbc-sources.mjs`, `fix-dsh-brief.mjs`, `fix-remaining-blocks.mjs`, `fix-validated-blocks.mjs`	One-off content fixes

> See `ANALYSIS_CONTENT_STANDARDS.md`, `CONTENT_CORRECTIONS.md`, `VALIDATION_TRIAGE.md` for the editorial rules these scripts enforce.

5. Academy & Supabase scripts

Script	Purpose
`seed-courses.mjs` (`npm run seed:courses`)	Seed course structure
`seed-all-courses.mjs`, `seed-courses-tier2.mjs`	Seed additional course tiers
`seed-courses.ts`, `seed-content.ts`, `seed-supabase.js`	Seed Supabase content
`link-sanity-slugs.mjs`	**Set `lessons.sanity_slug`** so rich Sanity bodies render
`audit-courses.mjs`	Validate course/track/lesson integrity & find orphans
`add-hie-intro-lesson.mjs`, `remove-welcome-lesson.mjs`, `restore-welcome-standalone.mjs`	Lesson membership edits
`delete-redundant-academy-modules.mjs`	Remove dupe modules (⚠️)
`sync-embeddings.ts`	Sync content embeddings into pgvector
`seed-hospitals.ts`	Seed hospital data
`reset-database.js`	⚠️ **Reset the database — destructive**

Backend Academy generator: `frontend/scripts/generate_course5.py`, `merge-expansions.py`.

6. Backend data loaders

Run from `backend/` with the venv active (`python scripts/<name>.py`):

Script	Purpose
`load_cms_hospitals.py`	Load CMS hospital dataset
`load_cms_quality_scores.py`	Load CMS quality scores
`sync_hospitals.py`	Sync hospital records
`sync_hti_scores.py`	Sync Health Tech Index scores

7. Media & narration (Piper TTS)

Repo-root `scripts/`:

- `generate-narration-piper.sh` — neural TTS via **Piper** (local, MIT-licensed, no API keys). Reads `.txt` files and writes one `.m4a` per file to `frontend/public/audio/narration/` (WAV → AAC/M4A via ffmpeg, ~10× smaller). Uses `.piper-venv` + `.piper-voices`.

```
./scripts/generate-narration-piper.sh            # all
        chapters
./scripts/generate-narration-piper.sh chapter-3  # one file
```

- `generate-narration-audio.sh` — older fallback narration generator.

Narration powers the book listen experience (`/book/listen`, `BookListenPlayer.tsx`) and the platform's voice/TTS layer (`frontend/lib/narration.ts`, `VoiceContext.tsx`, `VoiceFab.tsx`, activated with ⌘⇧V).

8. Document generation (PDF/DOCX/PPTX)

Repo-root tooling produces the marketing/whitepaper/deck artifacts (the `*.pdf`, `*.docx`, `*.pptx` files at repo root):

Script	Produces
`generate_pptx.py`	`COMBINED_DECK.pptx` and slide decks from the `*_DECK.md` sources
`generate_whitepaper_docx.py`	`HTR_WHITE_PAPER.docx` from `HTR_WHITE_PAPER.md`
`scripts/legacy-python/convert.py`, `merge_to_word.py`, `merge_images.py`	Markdown→Word / image merge utilities
`scripts/legacy-python/digest_latest.py`, `digest_critical.py`	Build content digests

To render this documentation set to PDF (for the "35-page" deliverable), any markdown→PDF tool works; e.g. with pandoc:

```
cd frontend/docs/platform-documentation
pandoc 0*.md 1*.md -o Vermont-Health-Platform-Documentation.pdf
       \
  --toc --pdf-engine=xelatex -V geometry:margin=1in
```

(The repo already produces `.md.pdf` companions for many root docs, so a markdown→PDF pipeline is established.)

9. How to run any script safely

1. `cd /Users/baba/Vermont-Health-Platform/frontend` (most content scripts) **or** `backend/` with the venv.
2. Confirm `frontend/.env.local` points at the intended Sanity dataset / Supabase project.
3. For destructive scripts, first **read the script** and dry-run / log target IDs (most print what they'll touch).
4. Run: `node scripts/<name>.mjs` (or `node scripts/<name>.js`, `npx tsx scripts/<name>.ts`, `python scripts/<name>.py`).
5. Verify in the app and/or in Sanity Studio / Supabase before moving on.

Continue to → 08 — Operations, Deployment & Maintenance

08 — Operations, Deployment & Maintenance

> **Verified against:** `.github/workflows/{ci,fly-deploy}.yml`, `vercel.json`, `backend/{fly.toml,railway.toml,Procfile,Dockerfile}`, `frontend/app/api/health/route.ts`, `frontend/app/api/cron/digest/route.ts`.

This document covers deploying the platform, the CI/CD pipeline, monitoring, routine maintenance, upgrades, backups, and incident response.

Table of contents

---

1. Deployment topology

Component	Platform	Region	Trigger
Frontend (Next.js)	**Vercel**	`iad1`	Auto-deploy on push to `main` (Vercel Git integration)
Backend (FastAPI)	**Fly.io** (app `vhp-backend`)	`sjc`	GitHub Action on push to `main` touching `backend/**`
Database	**Supabase Cloud**	—	Migrations applied manually / via CLI
CMS	**Sanity Cloud**	—	Studio publishes; webhook drives ingest

Backend has fallback descriptors for **Railway** (`railway.toml`, nixpacks, `/health` healthcheck) and a generic **Procfile**, plus a `Dockerfile` (python:3.11-slim, venv, uvicorn on 8000). Fly is the active target.

2. CI/CD pipeline

`.github/workflows/ci.yml` runs on push/PR to `main`. Jobs:

1. **frontend** — `npm ci` → `npm run lint` → `tsc --noEmit` → `npm run build` → `npm run bundle:check`. Builds with minimal public env from GitHub secrets; Sentry source-map upload skipped (`SENTRY_AUTH_TOKEN=""`).
2. **e2e** — needs `frontend`. Builds with `ALLOW_AUTH_BYPASS=true`, starts `npm run start`, waits on `localhost:3000`, runs Playwright (`frontend/e2e/`). Uploads the report artifact on failure.
3. **backend** — Python 3.11, `pip install`, `ruff check`, `mypy` (non-blocking), `pytest` (non-blocking until the suite matures).
4. **deploy-notify** — on `main` push after frontend+backend+e2e pass: echoes readiness. Vercel and Fly handle the actual deploys.

`.github/workflows/fly-deploy.yml` — on push to `main` under `backend/**`: `flyctl deploy` from `backend/` using `FLY_API_TOKEN`.

> After a backend deploy, **run `POST /api/ingest`** to rebuild the RAG index if content/schema changed (the CI notify step reminds you of this).

3. Deploying the frontend (Vercel)

- **Auto:** push to `main` → Vercel builds and deploys.
- **Build config** (`vercel.json`): `installCommand: npm install`, `buildCommand: cd frontend && ../node_modules/.bin/next build`, `outputDirectory: frontend/.next`, framework `nextjs`, region `iad1`.

- **Headers:** CORS headers on `/api/(.*)`.
- **Env:** set all `NEXT_PUBLIC_*` and server secrets in the Vercel project settings (mirror `frontend/.env.production.example`). Production secrets are **not** in the repo.
- **Manual deploy:** `vercel --prod` from repo root (or redeploy from the Vercel dashboard).

> **ISR / revalidation:** content is cached. The daily cron (`/api/cron/revalidate` in `vercel.json` crons, `0 0 * * *`) refreshes. For an immediate content refresh after a Sanity publish, trigger revalidation or redeploy.

4. Deploying the backend (Fly.io)

- **Auto:** push to `main` touching `backend/**` → GitHub Action runs `flyctl deploy`.
- **Manual:** `cd backend && flyctl deploy`.
- **Config** (`fly.toml`): app `vhp-backend`, region `sjc`, process `uvicorn main:app --host 0.0.0.0 --port 8000`, `force_https`, **auto-stop/auto-start machines**, `min_machines_running = 0` (scale-to-zero), 1 GB / 1 CPU.
- **Secrets:** `flyctl secrets set KEY=value` for every backend env var (Supabase, Sanity, LLM keys, `INGEST_SECRET`, `SENTRY_DSN`). Never bake secrets into the image.
- **Health:** `GET /health` (also Railway healthcheck path, 30s interval).

> **Cold starts:** scale-to-zero means the first request after idle is slow. If users complain about first-message latency, set `min_machines_running = 1`.

5. Database migrations in production

1. Add a new file `supabase/migrations/0NN_description.sql` (never edit a shipped one).
2. Test locally against a dev/staging Supabase or a throwaway DB.
3. Apply to production:

```
supabase db push                          # if CLI-linked
# or
psql "$SUPABASE_DB_URL" -f
        supabase/migrations/0NN_description.sql
```

4. If the migration changes content that RAG indexes, run `POST /api/ingest` afterward.

5. Watch Sentry + `/admin/analytics` for regressions.

6. Scheduled jobs / crons

Job	Where defined	Schedule	Does
Revalidate content	`vercel.json` crons → `/api/cron/revalidate`	`0 0 * * *` (daily)	Refres from S
Email digest	`frontend/app/api/cron/digest/route.ts` (+ `/api/digest`, `/api/digest/preview`)	(scheduled externally / cron)	Build email Loops users (`dig`
RAG log pruning	Supabase (migration 020)	DB-side	Trim `rag_`
pgvector HNSW maintenance	Supabase (migration 021)	DB-side	Keep index

7. Monitoring & health

Signal	Where
Frontend errors	Sentry (`@sentry/nextjs`, `NEXT_PUBLIC_SENTRY_DSN`)
Backend errors	Sentry (`sentry-sdk[fastapi]`, `traces_sample_rate=0.1`)
Backend liveness	`GET /health` → `{ index_ready }`; proxied by frontend `/api/health` → `{ ok, indexReady }`
In-app system status	`/system-vitals`, `BackendStatus.tsx`, `TickerStrip`
RAG quality	`rag_query_log`, `rag_feedback` tables
Usage/revenue	`/admin/analytics`, `/admin/revenue`
Web vitals	`WebVitalsReporter.tsx` → `web-vitals`

8. Routine maintenance checklist

Weekly

- Review Sentry for new error clusters (frontend + backend).
- Skim `rag_feedback` 👎 to find weak AI answers; re-ingest or fix source content.

- Confirm the digest cron sent (check Loops + logs).

Monthly

- Take a Sanity dataset export → `sanity-backups/` (see §10).
- Review Supabase storage growth (certs, audio) and DB size.
- Rotate any API keys nearing policy age (`/account/api-keys`, migration 023 supports rotation).
- `npm audit` / dependency review.

Per content release

- Run the relevant audit script (`audit-analysis-length.mjs`, `audit-courses.mjs`).
- After bulk content changes, run `POST /api/ingest` to rebuild RAG.
- Verify a sample page + a sample lesson render correctly.

9. Upgrades (dependencies, framework, content)

Frontend dependency bump

1. Update `frontend/package.json` (and root `package.json` if hoisted).
2. `cd frontend && npm install && npm run smoke` (typecheck + lint + build + bundle).
3. Run Playwright (`npm run test:e2e`).
4. Watch for: Next 16 / React 19 breaking changes, Tailwind v4 beta churn, Sanity/`next-sanity` majors.

Backend dependency bump

1. Update `backend/requirements.txt` (versions are pinned to the working venv).
2. Recreate venv, `pip install -r requirements.txt`, run app + `/health`.
3. Watch LlamaIndex core/integration version compatibility (it moves fast) and Groq/OpenAI/Anthropic SDK changes.

LLM model upgrade

- Change `GROQ_MODEL` / `MODEL_*` constants in `backend/config.py` (or env). The fallback chain in `services/llm.py` insulates against a bad model. Re-test chat for each role.

Content upgrade — follow the Academy one-lesson-at-a-time workflow (Doc 05) and the Analysis standards (Doc 03); always re-ingest after.

> The repo's `UPGRADE_PLAN.md` and `PLAN_SANITY_ECOSYSTEM.md` are historical planning docs — useful context, but treat this section as the current procedure.

10. Backups & disaster recovery

Asset	Backup method	Restore
Sanity content	`npx sanity dataset export production <file>.tar.gz` → `sanity-backups/`	`npx sanity dataset import <file>.tar.gz production --replace` (⚠️ overwrites)
Supabase DB	Supabase automated backups + manual `pg_dump "$SUPABASE_DB_URL"`	`psql` restore / Supabase PITR
Supabase Storage	Bucket export	Re-upload
RAG index	*Not backed up — it's derived*	Rebuild with `POST /api/ingest` from Sanity
Code	Git (`main`)	Redeploy from a known-good commit

Recovery principle: Sanity + Supabase are the only stateful sources of truth. The RAG index, the built frontend, and the running backend are all reproducible. Restore content + DB, redeploy code, re-ingest.

11. Incident runbook

Incident	First checks	Action
Site down	Vercel status, recent deploy, build logs	Roll back to last green deploy in Vercel
AI Analyst failing	`/api/health` → `indexReady` ; backend Sentry; Fly machine status	Restart Fly machine; if index empty, `POST /api/ingest`
Payments failing	Stripe dashboard; `/api/stripe/webhook` logs; `stripe_events`	Replay missed webhook events; reconcile `subscriptions`
Content not updating	ISR cache; was it published in Sanity?	Trigger `/api/cron/revalidate` or redeploy
RAG drifted from content	Compare a known Sanity doc vs an AI answer	`POST /api/ingest` full rebuild
Login broken	Supabase Auth status; JWT secret parity	Verify `SUPABASE_JWT_SECRET` matches across frontend/backend
Backend cold-start latency	Fly `min_machines_running`	Set to 1

Continue to → 09 — User Guide & Usability

09 — User Guide & Usability

> **Verified against:** `frontend/app/*` routes, `frontend/components/{CommandPalette,VoiceContext,VoiceFab,Header,HomeSidebar,RightSidebar,AppShell}.tsx`, `frontend/app/pricing/page.tsx`, `frontend/app/research-lab/*`. Companion: `training/` user-guides & feature-guides.

This is the end-user manual. It explains what the platform does and how to use every major feature, written for clinicians, executives, policy analysts, investors, and researchers.

Table of contents

---

1. Getting started

1. **Sign up** at `/signup` (or `/login` if you have an account). Auth is email/password via Supabase.
2. **Onboarding** (`/onboarding`, `/welcome`, `/start`) — pick your role/interests so content and personalized learning are tailored.
3. **Beta access:** during pre-launch, an access code may be required (`/beta`).
4. **Explore** from the home page — the left **HomeSidebar** is your primary navigation; the right sidebar is the **AI Analyst**.

2. Plans & access tiers

The platform offers these tiers (see `/pricing`):

Tier	Who it's for	Unlocks
Free	Anyone	Public articles, limited features
Student	Learners	Academy + discounted access
Subscriber	Professionals	Full content + AI Analyst (RAG)
Professional	Power users	Agentic AI (tool-using), deeper data
Advisory	Consulting clients	Advisory hub, premium AI (Claude), reports

Manage your plan at `/account/subscription` and `/account/billing` (Stripe-powered, including team checkout). Upgrade prompts appear on gated content.

3. Navigating the app

- **Header** (`Header.tsx`) — global search, account, theme toggle.
- **HomeSidebar** (left) — sidebar-first navigation into the six pillars and major sections. This is the primary nav.
- **RightSidebar** (right) — the AI Analyst widget, available on every page; expand to full chat.
- **TickerStrip** — a live scrolling band of "System Vitals" (ER wait time, etc.), driven by Sanity `ticker` docs.
- **Command Palette** — press **⌘K** (or Ctrl+K) to jump anywhere by name.
- **Breadcrumbs**, **Site Map** (`/site-map`), and **Search** (`/search`) help you orient.

4. The Six-Pillar sections

The platform's content is organized into six pillars, each a top-level section with overview, subpages, and `[slug]` articles:

Pillar	Section	Example subpages
Policy	`/policy`	regulation, mandates, global, feasibility
Economics	`/economics`	value, market, investment, cea
Technology	`/technology`	ai, digital, security, workflow
Clinical	`/clinical`	hah (hospital-at-home), precision, virtual, population, genomics
Equity	`/equity`	sdoh, bias, access
Operations	`/operations`	revenue-cycle, supply-chain, workforce, compliance, payer-network

Each pillar surfaces **Analyses** (research briefs), case studies, and "From the Book" cross-links.

5. The AI Analyst

Your research co-pilot. It answers questions grounded in the platform's content (RAG) and cites its sources.

- **Where:** the right-sidebar widget on any page, or the full page at `/chat`.
- **How to use:** type a question (e.g. "How does Vermont's Act 167 change hospital budgeting?"). The Analyst retrieves relevant Analyses/case studies, answers with citations, and may link you to the right interactive tool.
- **Follow-ups:** suggested next questions appear after each answer.
- **Tiers:** Subscribers get grounded RAG answers. Professional/Advisory/Admin get the **agentic** Analyst, which can call live data tools (state metrics, Vermont bed capacity, best-transfer routing, etc.).
- **Feedback:** rate answers 👍/👎 — this feeds answer-quality monitoring.

> The Analyst only knows what's published on the platform. If an answer seems thin, the underlying content may not be indexed yet.

6. The Academy

Structured learning at `/academy`.

- **Courses** (`/academy/courses`) — multi-module courses across the six pillars; some are cohort-based, some self-paced.
- **Tracks & Modules** (`/academy/tracks`, `/academy/modules`) — curated learning paths.
- **Lessons** — rich, illustrated lessons (key concepts, real-world examples, stat cards, comparisons, warnings) with optional audio narration.

- **Quizzes** — check your understanding; attempts and scores are saved.
- **Case Studies, Faculty, Glossary, Webinars** — supporting resources.
- **Personalized Learning** (`/academy/personalized-learning`) — an AI-generated path tailored to your role and goals, optionally with audio.
- **Progress & Certificates** — track completion at `/account/courses`; earn a **certificate** on completion (verifiable publicly at `/verify/[hash]`).

7. Dashboards, states & simulators

Interactive, data-driven tools:

- **State profiles** (`/states/[state]`) — Rural Health Transformation program profiles, awards, initiatives, metrics.
- **State Performance Index** — a 0–100 six-pillar composite per state.
- **Compare states** (`/compare-states`, `/dashboard/compare`).
- **Simulators** — model policy and program outcomes:
  - Vermont: `/vermont-act-167`, `/vermont-act-68`, `/vermont-rht-program`, `/vermont-blueprint`, `/vermont-medicaid`, `/vermont-sash`, plus `/htr-simulator`, `/impact-simulation`.
  - Other states: `/california-calaim/simulator`, `/oregon-cco/simulator`.
  - Eligibility: `/medicaid-eligibility-simulator`.
- **Indices & trackers** — `/htr-index`, `/hti-dashboard`, `/investment-tracker`, `/transformation-friction-index`, `/bed-capacity`.

8. The Research Lab

`/research-lab` — seven specialized workspaces for deeper analysis:

Workspace	Focus
`technology-ai`	AI & technology
`payment-models`	Payment & reimbursement
`policy-quality`	Policy & quality
`population-equity`	Population health & equity
`vbc-clinical-quality`	Value-based / clinical quality
`interoperability`	Data interoperability
`knowledge-workspace`	A working/knowledge surface

The AI Analyst can point you to the right Lab tool for a given question.

9. The Wire, the Book & media

- **The Wire** (`/the-wire`) — a live feed of healthcare-transformation news/insights with threaded comments.
- **The Book** — *Transforming American Healthcare*: read at `/book` , listen at `/book/listen` (Piper-narrated audio), with chapter notes and "From the Book" callouts woven through pillar pages.
- **Media** (`/media` , `/media/videos` , `/multimedia`) — video library and multimedia.
- **Trending / Daily Insight** — the dark insight strip surfaces the current "Quote/Stat/Chart of the Day."

10. Connect & Community

- **Community** (`/community`) — forums, threads, upvotes.
- **Connect hub** (`/connect`) — directory, ask-an-expert, alerts, toolkits, office hours, forums.
- **Advisory hub** (`/advisory`) — for consulting clients: services, regulatory, financial audit, independent review, reports, training.
- **Survey** (`/survey`) — periodic editions; aggregated results at `/survey/results` .

11. Voice & accessibility

- **Voice input/output:** toggle the mic with ⌘⇧**V** (hide/show the voice FAB with ⌘⇧**H**). Ask the Analyst by voice; answers can be spoken (TTS).
- **Accessibility controls:** dark-mode toggle, font-size toggle, print/save-to-PDF buttons on articles, semantic headings, keyboard navigation.

12. Your account

`/account/*` :

Page	Purpose
`/account/profile`	Name, org, bio, avatar
`/account/subscription` / `/account/billing`	Manage plan & payment (Stripe portal)
`/account/courses`	Enrolled courses & progress
`/account/bookmarks`	Saved items (also `/saved`)
`/account/referrals`	Your referral code & rewards
`/account/api-keys`	Developer API keys (create/rotate/revoke)

13. Keyboard shortcuts

Shortcut	Action
⌘K / Ctrl+K	Open the Command Palette (jump anywhere)
⌘⇧V	Toggle voice mic
⌘⇧H	Hide/show the voice FAB
In Command Palette: `H` `E` `P` `T` `C` `Q`	Quick-nav Home, Economics, Policy, Technology, Clinical, Equity

> For role-specific quick-starts (clinician, executive, economist, policy analyst, investor/consultant, researcher, health-tech, compliance), see `training/user-guides/`. For feature deep-dives (Academy, AI Analyst, Research Lab, Voice), see `training/feature-guides/`.

Continue to → 10 — Reference Appendices

10 — Reference Appendices

> **Verified against:** `frontend/app/**`, `supabase/migrations/*`, `backend/config.py`, `frontend/.env.production.example`, `frontend/sanity/schemaTypes/*`.

Lookup tables for the whole platform: environment variables, API routes, page routes, Sanity schema, Supabase tables, and a glossary.

Table of contents

A. Environment variables

Frontend (`frontend/.env.local` / Vercel)

Var	Scope	Purpose
`NEXT_PUBLIC_APP_URL`	public	App base URL
`ALLOW_AUTH_BYPASS`	server	Dev/CI only — skip auth gating
`NEXT_PUBLIC_SUPABASE_URL`	public	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	public	Supabase anon client key
`SUPABASE_SERVICE_ROLE_KEY`	**server-only**	Privileged DB access
`NEXT_PUBLIC_SANITY_PROJECT_ID`	public	`fxz10xl7`
`NEXT_PUBLIC_SANITY_DATASET`	public	`production`
`NEXT_PUBLIC_SANITY_API_VERSION`	public	`2023-10-01`
`SANITY_API_TOKEN`	server	Sanity write token
`SANITY_WEBHOOK_SECRET`	server	Verify Sanity webhooks
`INGEST_SECRET`	server	Sanity→backend ingest bridge (matches backend)
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	public	Stripe.js
`STRIPE_SECRET_KEY`	server	Stripe API
`STRIPE_WEBHOOK_SECRET`	server	Verify Stripe webhooks
`STRIPE_PRICE_SUBSCRIBER_MONTHLY` / `_YEARLY`	server	Price IDs
`STRIPE_PRICE_STUDENT_MONTHLY` / `_YEARLY`	server	Price IDs
`STRIPE_PRICE_PROFESSIONAL_MONTHLY` / `_YEARLY`	server	Price IDs
`LOOPS_API_KEY`	server	Email
`LOOPS_TEMPLATE_WELCOME` / `_DIGEST` / `_PAYMENT_FAILED` / `_TRIAL_ENDING` / `_SURVEY_RESULTS`	server	Email templates
`API_KEY_HMAC_SECRET`	server	Hash developer API keys

Var	Scope	Purpose
`PYTHON_BACKEND_URL` / `BACKEND_URL`	server	Backend base URL for proxying
`NEXT_PUBLIC_SENTRY_DSN`	public	Sentry
`SENTRY_ORG` / `SENTRY_PROJECT` / `SENTRY_AUTH_TOKEN`	server	Sentry source maps

Backend (`backend/.env` / Fly secrets) — from `backend/config.py`

Var	Default	Purpose
`GROQ_API_KEY`	—	Default LLM
`ANTHROPIC_API_KEY`	—	Advisory/admin LLM (optional)
`OPENAI_API_KEY`	—	Embeddings + last-resort LLM
`GROQ_MODEL`	`llama-3.3-70b-versatile`	Subscriber model
`SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_URL`	—	DB
`SUPABASE_SERVICE_ROLE_KEY`	—	Privileged DB
`SUPABASE_JWT_SECRET`	—	Verify session JWTs
`SUPABASE_DB_URL`	—	Direct Postgres (pgvector)
`SANITY_PROJECT_ID`	—	Pull content for ingest
`SANITY_DATASET`	`production`	Dataset
`SANITY_API_TOKEN`	—	Read content
`SANITY_API_VERSION`	`2023-10-01`	API version
`FRONTEND_URL`	`http://localhost:3000`	CORS allow-list
`INGEST_SECRET`	—	Ingest auth (matches frontend)
`SENTRY_DSN`	—	Monitoring
`ENVIRONMENT`	`production`	Sentry environment tag

(Also referenced in `config.py`: `MODEL_FREE`, `MODEL_SUBSCRIBER`, `MODEL_ADVISORY`, `EMBEDDING_MODEL`, `MAX_SYSTEM_PROMPT_LEN`.)

B. Next.js API routes (`/api/*`)

`frontend/app/api/**/route.ts`:

Route	Purpose
`academy/certificates`	Issue/return a course certificate (idempotent)
`admin/beta-codes`	Manage beta access codes
`admin/users`	Admin user management
`beta/verify`, `beta/clear`	Beta gate
`bookmarks`	CRUD bookmarks
`chapter-notes`	Book chapter notes
`chat`	Proxy to backend RAG chat
`suggest`	Follow-up suggestions
`cron/digest`, `digest`, `digest/preview`	Email digest
`directory`	Member directory
`feedback`	Feedback capture
`health`	Proxy backend `/health`
`hospitals`	Hospital data
`hti-scores`	Health Tech Index
`keys/create`, `keys/revoke`, `keys/rotate`	Developer API keys
`learning-paths`	Learning paths
`loops/welcome`	Loops welcome email
`personalized-learning`, `personalized-learning/audio`	Personalized learning
`rht-states`	RHT state data
`role-content`	Role-gated content
`search`	Search
`state-metrics`	State metrics
`stripe/checkout`, `stripe/portal`, `stripe/team-checkout`, `stripe/webhook`	Billing
`subscribe`	Newsletter subscribe
`tester-report`	Tester reports
`webhooks/sanity`	Sanity→backend ingest bridge
`wire`, `wire/comments`, `wire/comments/[id]`, `wire/comments/counts`	The Wire + comments

C. Backend (FastAPI) endpoints

Endpoint	Auth	Purpose
`GET /health`	open	Liveness + `index_ready`
`POST /api/chat`	subscriber+	Streaming RAG/agentic chat
`POST /api/suggest`	open	Follow-up questions
`POST /api/ingest` (202)	`INGEST_SECRET`	Full re-index job
`GET /api/ingest/status`	`INGEST_SECRET`	Job status
`POST /api/ingest/webhook` (202)	Bearer `INGEST_SECRET`	Incremental ingest
`GET /api/v1/states`	API key	All state profiles
`GET /api/v1/states/{state_id}`	API key	One state profile
`GET /api/v1/survey/results`	API key	Aggregated survey
`GET /vermont/bed-capacity`	subscriber+	Bed capacity
`PATCH /vermont/bed-capacity/{hospital_id}`	subscriber+	Update capacity
`GET /vermont/alerts`	subscriber+	Capacity alerts
`POST /vermont/transfer-log`	subscriber+	Log a transfer
`GET /vermont/transfer-log`	subscriber+	Read transfers
`/api/personalized-learning*`	subscriber+	Personalized path generation

D. Page routes (full sitemap)

> ~160 page routes. Dynamic segments shown as `[param]`.

Top-level / marketing: `/`, `/about`, `/about/framework`, `/about/methodology`, `/mission`, `/values`, `/pricing`, `/subscribe`, `/upgrade`, `/faq`, `/changelog`, `/developers`, `/privacy`, `/terms`, `/site-map`, `/search`, `/setup`, `/welcome`, `/start`, `/onboarding`, `/tester`.

Auth: `/login`, `/signup`, `/forgot-password`, `/reset-password`, `/beta`, `/verify/[hash]`.

Six pillars: `/policy` (+ `/[slug]`, regulation, mandates, global, feasibility) · `/economics` (+ `/[slug]`, value, market, investment, cea) · `/technology` (+ `/[slug]`, ai, digital, security, workflow) · `/clinical` (+ `/[slug]`, hah, precision, virtual, population, genomics) · `/equity` (+ `/[slug]`, sdoh, bias, access) · `/operations` (+ `/[slug]`, revenue-cycle, supply-chain, workforce, compliance, payer-network).

Articles/reading: `/articles/[slug]`, `/read/[slug]`, `/library`.

Academy: `/academy`, `/academy/courses` (+ `/[slug]`), `/academy/tracks` (+ `/[courseSlug]` + `/[lessonSlug]`), `/academy/modules/[slug]`, `/academy/case-studies` (+ `/[slug]`), `/academy/faculty`, `/academy/glossary`, `/academy/webinars` (+ `/[slug]`), `/academy/getting-started` (+ `/research-lab`), `/academy/personalized-learning`, `/academy/medicaid` (+ `/glossary`).

Account: `/account`, `/account/{profile,billing,subscription,courses,bookmarks,referrals,api-keys}`, `/saved`.

Admin: `/admin`, `/admin/{users,analytics,revenue,access-codes,role-changes,ingest}`.

Advisory: `/advisory`, `/advisory-hub`, `/advisory/{approach,capability-assessment,consulting,contact,financial-audit,independent-review,it-consulting,regulatory,reports,research,services,training}`.

Dashboards / states / simulators: `/states` (+ `/[state]`), `/dashboard` (+ `/[state]` + `/hospitals` + `/hospitals/[hospital]`, `/compare`, `/simulator`, `/vermont/hospitals/...`), `/compare-states`, `/htr-index`, `/htr-simulator`, `/hti-dashboard`, `/impact-simulation`, `/investment-tracker`, `/transformation-friction-index`, `/bed-capacity`, `/ahead-model`.

Vermont-specific: `/vermont-act-167` (+ `/hospitals/[slug]`, `/simulator`), `/vermont-act-68` (+ `/simulator`), `/vermont-rht-program`, `/vermont-blueprint`, `/vermont-medicaid`, `/vermont-sash`, `/vermont-sdoh`, `/vermont-vcci`, `/vermont-designated-agencies`, `/vermont-legislative-resources`.

Other states: `/california-calaim` (+ `/simulator`), `/oregon-cco` (+ `/simulator`), `/medicaid-eligibility-simulator`.

Research Lab: `/research-lab` (+ `interoperability`, `knowledge-workspace`, `payment-models`, `policy-quality`, `population-equity`, `technology-ai`, `vbc-clinical-quality`).

Content/media/community: `/the-wire`, `/trending-topics`, `/book` (+ `/listen`), `/media/videos`, `/multimedia`, `/community` (+ `/[slug]`, `/new`, `/thread/[id]`), `/connect` (+ `alerts`, `apply`, `ask`, `directory`, `forums`, `register-office-hours`, `toolkits`), `/connect-hub`, `/survey` (+ `/results`, `/thank-you`), `/system-vitals`.

CMS: `/studio/[[...index]]` (embedded Sanity Studio).

E. Sanity schema types

22 types, registered in `frontend/sanity/schemaTypes/index.ts`:

`blockContent` (object), `category`, `post`, `author`, `policyAnalysis` (Analysis), `hospital`, `academyModule`, `caseStudy`, `course`, `webinar`, `report`, `ticker`, `dailyInsight`, `analystNote`, `instructor`, `definition`, `audio`, `rhtState`, `statePerformanceIndex`, `subscriber`, `investmentDeal`.

Field details in Doc 03 §3–§6.

F. Supabase tables

58 `CREATE TABLE` statements across 34 migrations. Grouped:

- **Identity/billing:** `profiles`, `user_roles`, `subscriptions`, `stripe_customers`, `stripe_events`, `role_change_log`, `beta_access_codes`.
- **Academy (legacy):** `course_enrollments`, `module_progress`, `certifications`, `learning_tracks`.
- **Academy (player):** `courses`, `tracks`, `lessons`, `quizzes`, `quiz_questions`, `quiz_options`, `audio_slots`, `course_player_enrollments`, `course_lesson_progress`, `course_quiz_attempts`, `learner_audio_uploads`, `lesson_bookmarks`, `lesson_notes`.
- **RAG:** `rag_documents`, `rag_query_log`, `rag_feedback`.
- **Content data:** `hospitals`, `hospitals_cms`, `hti_scores`, `state_health_metrics`, `state_initiatives`, `state_time_series`, `national_benchmark`, `rht_state_profiles`, `ticker_cache`.
- **Community/engagement:** `community_categories`, `community_threads`, `community_posts`, `community_upvotes`, `wire_comments`, `wire_comment_upvotes`, `bookmarks`, `chapter_notes`, `professional_profiles`.
- **Growth/ops:** `referral_codes`, `referral_events`, `api_keys`, `survey_editions`, `survey_responses`, `webhook_inbox`, `user_learning_paths`, `conversations`, `conversation_messages`.

- **Advisory:** `advisory_clients`, `advisory_reports`.

Schema details in Doc 04.

G. Glossary of project terms

Term	Meaning
HTR	Healthcare Transformation Roadmap — the internal name for the platform/brand
AI Brain	The FastAPI backend (`backend/`), title "HTR AI Brain"
Six-Pillar Framework	Policy, Economics, Technology, Clinical, Equity, Operations
Analysis	A research brief (`policyAnalysis` in Sanity)
The Wire	Live news/insight feed (`/the-wire`)
RHT	Rural Health Transformation (program) — `rhtState` profiles
HTI	Health Tech Index — `hti_scores`
RAG	Retrieval-Augmented Generation — the AI Analyst's grounding method
Course Player	The relational Academy model (migration 028+): courses→tracks→lessons
`sanity_slug`	The lesson column linking a Supabase lesson to its rich Sanity body
Ingest	The pipeline that copies published Sanity content into the RAG vector store
AHEAD model	A CMS state total-cost-of-care model (`/ahead-model`)
CalAIM / CCO	California / Oregon Medicaid transformation programs (simulators)
Act 167 / Act 68	Vermont legislation modeled by simulators

End of the Vermont Health Platform documentation set. Return to → Index.